

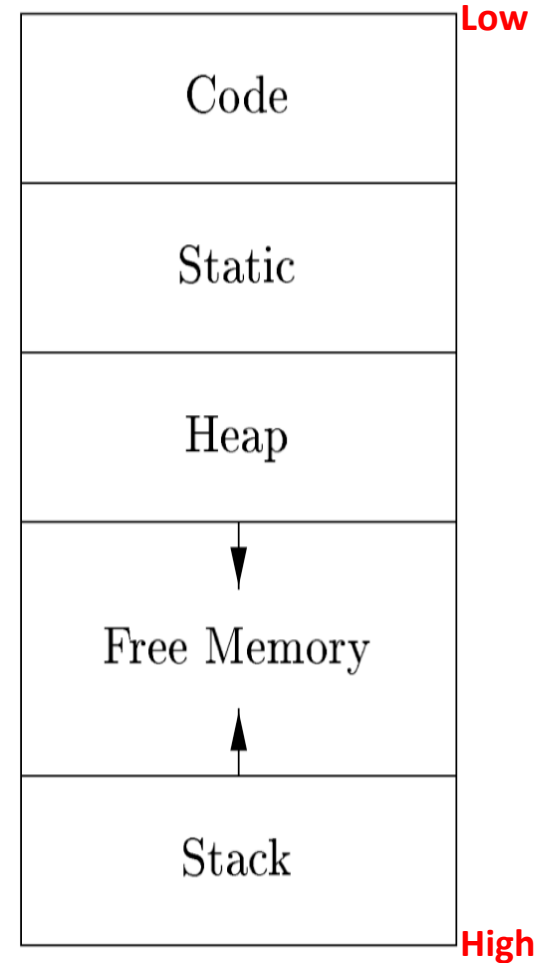
Garbage Collection

Chapter 13

Storage Organization

A typical way of subdividing memory is:

- **Code**: the executable target code
- **Static**: the data objects of which the size can be known at compile time, e.g.,
 - global constants
 - data generated by the compiler
- **Stack**: data structures called **activation records** that get generated during procedure calls
- **Heap**: data that is allocated and deallocated under program control
 - for C: *malloc*, *free*
 - for Java: *new*



Manual Memory Management

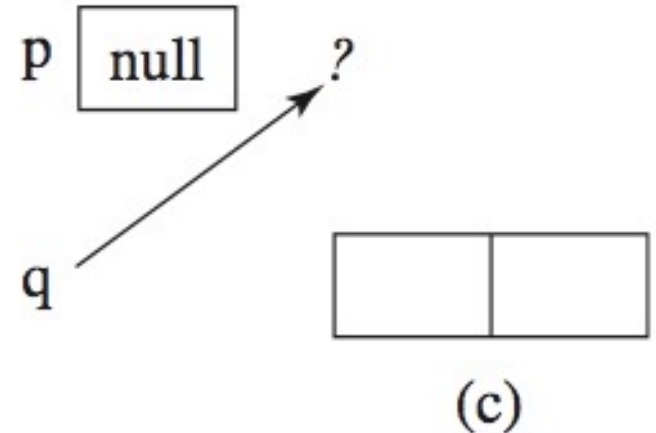
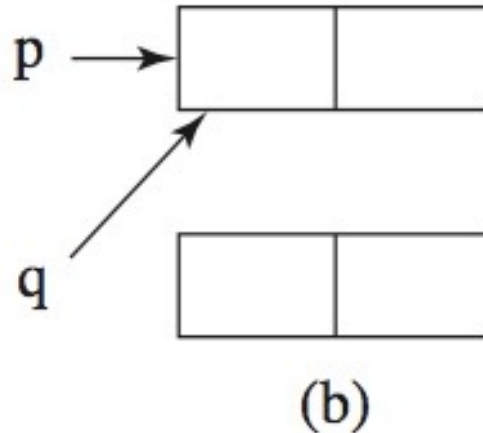
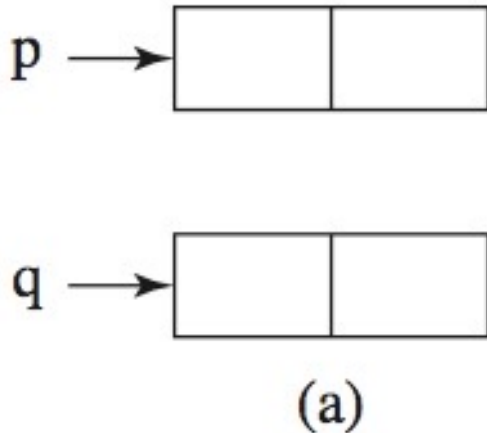
- Manual method
 - Approach used in C, C++
 - The use of `malloc` and `free` to perform dynamic allocation and de-allocation of pointer
- Problems with manual management
 - Easily lead to problems: memory leaks, double frees, use-after frees
 - Type-safe problems
- Storage bugs are hard to find
 - A bug can lead to a visible effect far away in time and program text from the source

Automatic Memory Management

- **Automatic Memory Management:** Reclamation of memory is automatic
- **Garbage:** Allocated but **no longer used** storage

```
class node {  
    int value;  
    node next;  
}  
node p, q;
```

```
p = new node();  
q = new node();  
q = p;  
delete p;
```



Garbage Collection: What

- **Garbage collection**: The process of reclamation of allocated but no longer used storage without an explicit call to free.
- Garbage collection is performed not by the compiler but by the runtime system (the support programs linked with the compiled code)

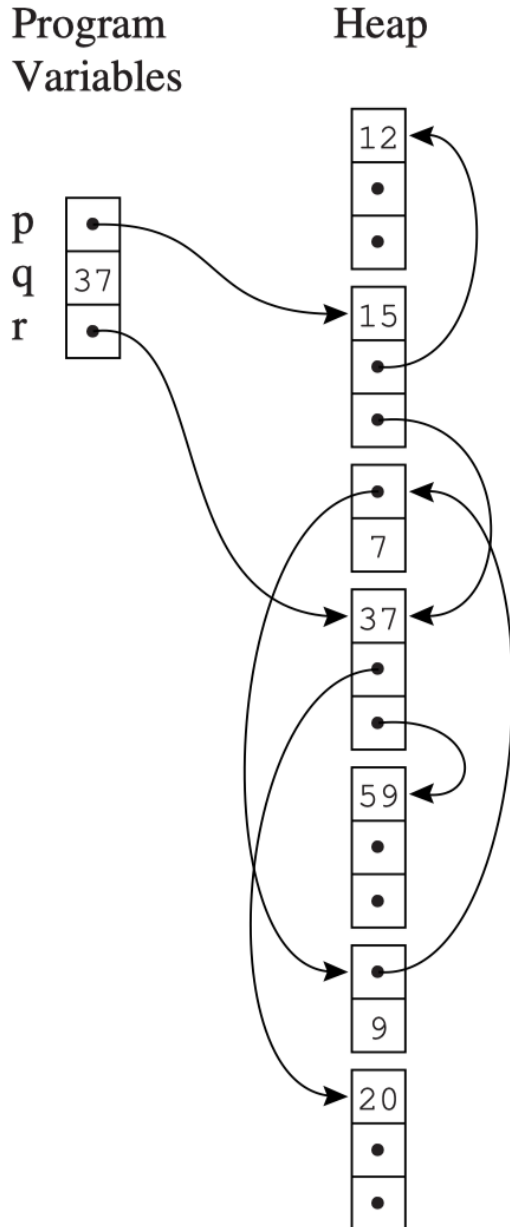
Garbage Collection: How

- **Garbage**: Allocated but **no longer used** storage
- Ideally, any record that is not dynamically live (will not be used in the future of the computation) is garbage.
- But it is **undecidable** whether an object is garbage
 - Need to rely on a conservative approximation
- **Idea**: Use reachability information as “approximation”
 - Heap-allocated records that are not **reachable** by any chain of pointers from program variables are **garbage**
 - Conservative: **not reachable** -> **garbage**
- **Garbage** -> **may be reachable**

Garbage Collection: How

- An object x is **reachable** if and only if
 - A register contains a pointer to x , or
 - another reachable object y contains a pointer to x

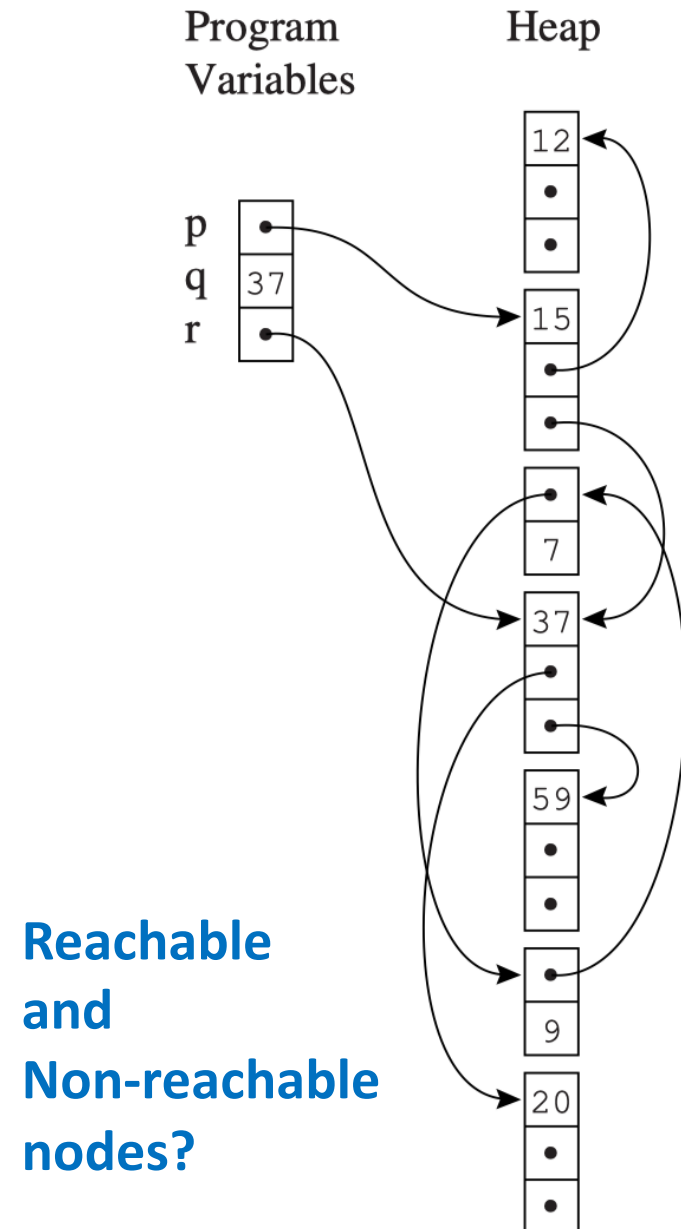
Basic Data Structure: Directed Graph



- Program variables and heap-allocated records form a **directed graph**.
 - **Nodes**
 - **Edges**: pointing relationship
- The program variables are the **roots** of this graph.
 - Registers
 - Local vars/formal parameters on stack
 - global variables
- A node **n** is **reachable** if there is a path of directed edges **r -> ... -> n**
 - **r**: some root

Garbage Collection: Example

```
let
  type list = {link: list, key: int}
  type tree = {key: int, left: tree,
right: tree}
  function maketree() = ...
  function showtree(t: tree) = ...
in
  let var x := list{link=nil, key=7}
    var y := list{link=x, key=9}
  in x.link := y
  end;
  let var p := maketree()
    var r := p.right
    var q := r.key
  in garbage-collect here
    showtree(r)
  end
end
```



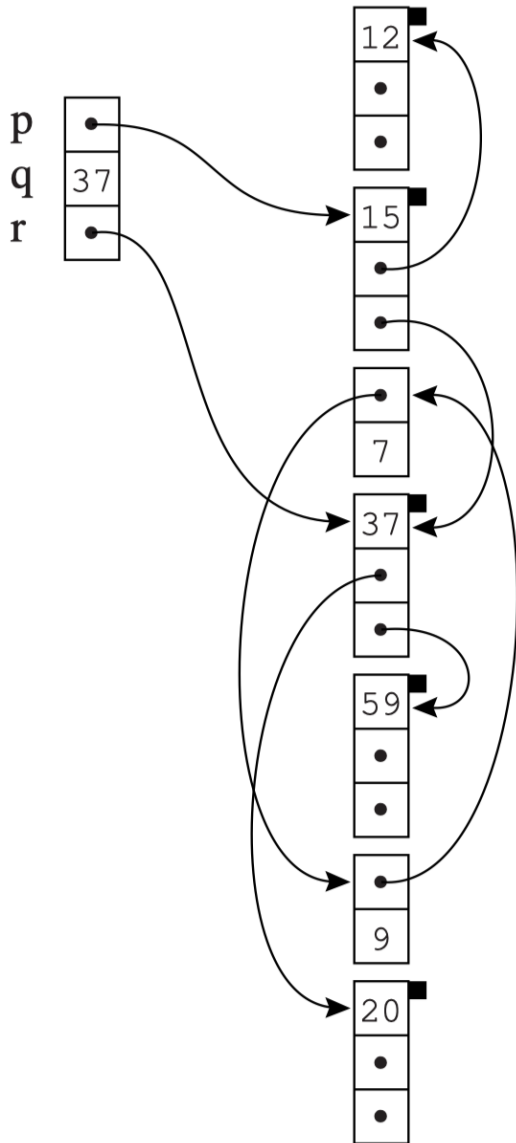
Outline

- Mark-and-Sweep Collection
- Reference Counts
- Copying Collection
- Interface to the Compiler

Outline

- **Mark-and-Sweep Collection**
- Reference Counts
- Copying Collection
- Interface to the Compiler

Mark-and-Sweep Collection



(a) *Marked*

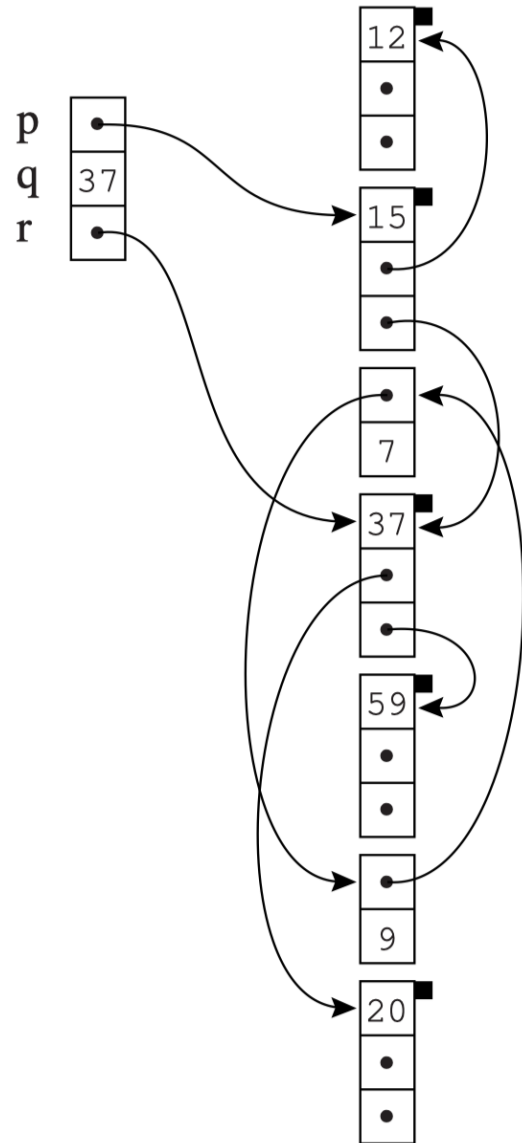
- **Mark:**
 - Search the graph from the roots (program variables)
 - Mark the nodes that are visited
- A graph-search algorithm such as **depth-first search** can **mark** all the reachable nodes.

```

function DFS(x)
if x is a pointer into the heap
    if record x is not marked
        mark record x
        for each field fi of record x
            DFS(x.fi)

```

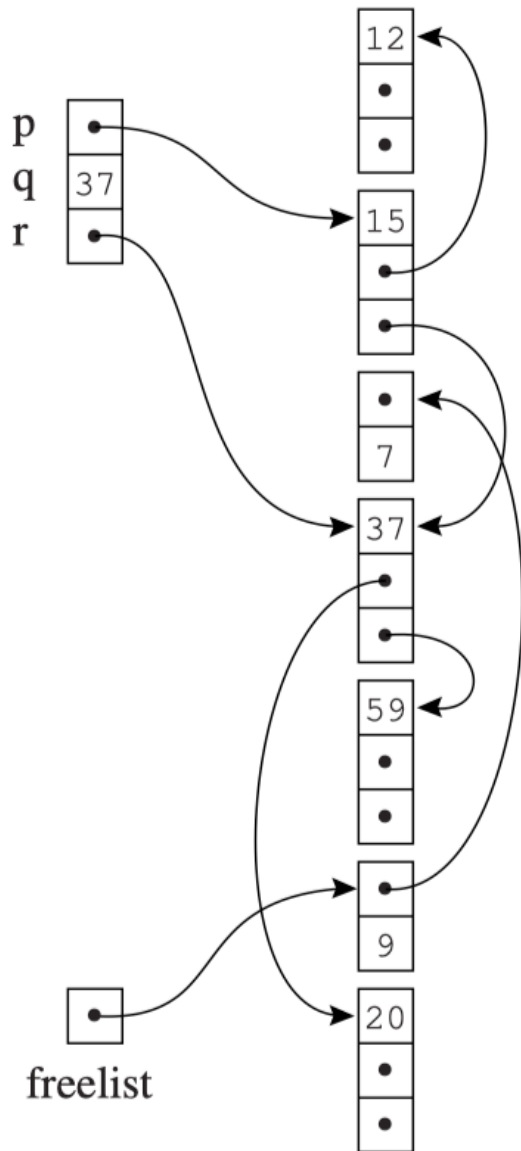
Mark-and-Sweep Collection



(a) Marked

- Any record not marked must be **garbage**, and should be **reclaimed**.
- How?

Mark-and-Sweep Collection



(b) Swept

- **Sweep:**

- Sweep the entire heap by a linear scan
- Link the unmarked nodes in a linked list (*freelist*)
- Unmark marked nodes for the next garbage collection

Mark-and-Sweep Collection - Algorithm

Mark phase:

for each root v
 DFS(v)

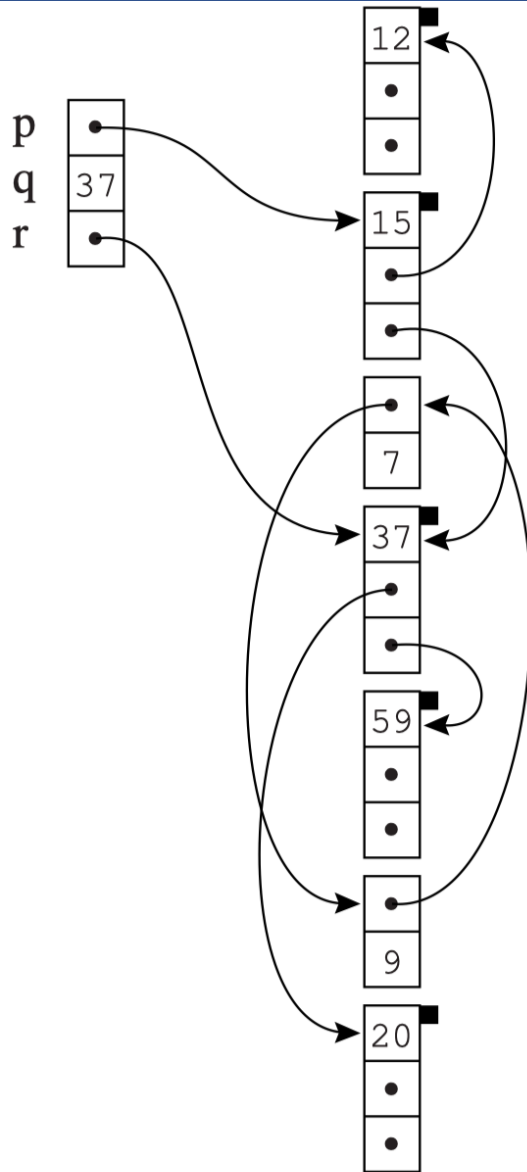
Sweep phase:

$p \leftarrow$ first address in heap
while $p <$ last address in heap
 if record p is marked
 unmark p
 else let f_1 be the first field in p
 $p.f_1 \leftarrow$ freelist
 freelist $\leftarrow p$
 $p \leftarrow p + (\text{size of record } p)$

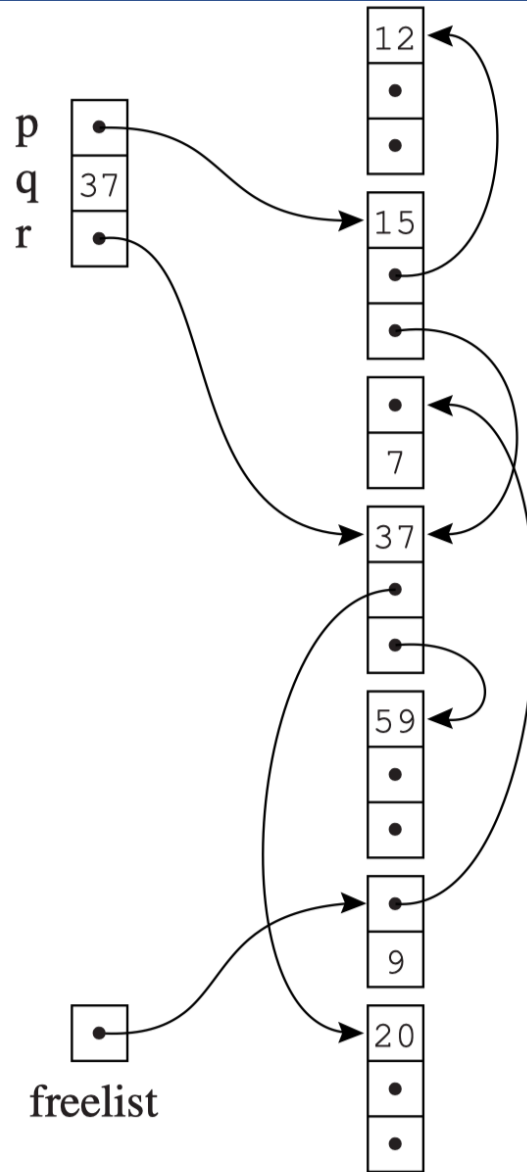
ALGORITHM 13.3. Mark-and-sweep garbage collection.

- After the garbage collection, the compiled program resumes execution.
- Whenever it wants to heap-allocate a new record, it gets a record from the *freelist*
- Do *garbage collection again* when *freelist* is empty.

Mark-and-Sweep Collection



(a) Marked



(b) Swept

- Mark
- Sweep
 - freelist
 - unmark

Cost of Mark-and-Sweep Collection

- H : heap size, R : reachable data
- **Time of GC**
 - **Mark**: DFS takes time proportional to R
 - **Sweep**: takes time proportional to H .
 - Total time: $c_1R + c_2H$
- GC replenish the freelist with $H-R$ words
- Amortized cost $(c_1R + c_2H)/(H - R)$
 - If R is closed to H , the cost is very High

The Problem of DFS

- The DFS algorithm is **recursive**
 - H : heap size
 - Extreme case: H stack frames form an H -element linked list
 - The length of the stack of activation records for DFS would be larger than the entire heap.
- **Solution:** Use an **explicit stack** instead of recursion
 - H words instead of H activation records

Using an Explicit Stack

```
function DFS(x)
  if  $x$  is a pointer and record  $x$  is not marked
    mark record  $x$ 
     $t \leftarrow 1$ 
     $\text{stack}[t] \leftarrow x$  // push the start of DFS on stack
    while  $t > 0$ 
       $x \leftarrow \text{stack}[t]; t \leftarrow t - 1$  // pop an item from the stack
      for each field  $f_i$  of record  $x$ 
        if  $x.f_i$  is a pointer and record  $x.f_i$  is not marked
          mark  $x.f_i$ 
           $t \leftarrow t + 1; \text{stack}[t] \leftarrow x.f_i$ 
```

- t : the index of the stack top
- stack : a worklist

Using an Explicit Stack

```
function DFS(x)
```

```
  if  $x$  is a pointer and record  $x$  is not marked
```

```
    mark  $x$ 
```

```
     $t \leftarrow 1$ 
```

```
    stack[ $t$ ]  $\leftarrow x$  // push the start of DFS on stack
```

```
    while  $t > 0$ 
```

```
       $x \leftarrow \text{stack}[t]; t \leftarrow t - 1$  // pop an item from the stack
```

```
      for each field  $f_i$  of record  $x$ 
```

```
        if  $x.f_i$  is a pointer and record  $x.f_i$  is not marked
```

```
          mark  $x.f_i$ 
```

```
           $t \leftarrow t + 1; \text{stack}[t] \leftarrow x.f_i$ 
```

- Benefit: H words instead of H activation records
- However, it is still unacceptable to require auxiliary stack memory as large as the heap being collected.

Pointer Reversal

- **Problem:** DFS needs a stack and the depth of the stack could be as large as the heap size
- Can we implement DFS without using an explicit stack?
- **Basic Idea:** store the DFS stack in the directed graph itself

```
function DFS(x)
    ...                // initialization
    while t > 0
        ...            // pop an item from the stack
        for each field fi of record x
            if x. fi is a pointer and record x.fi is not marked
                mark x. fi
                t ← t + 1; stack[t] ← x. fi
```

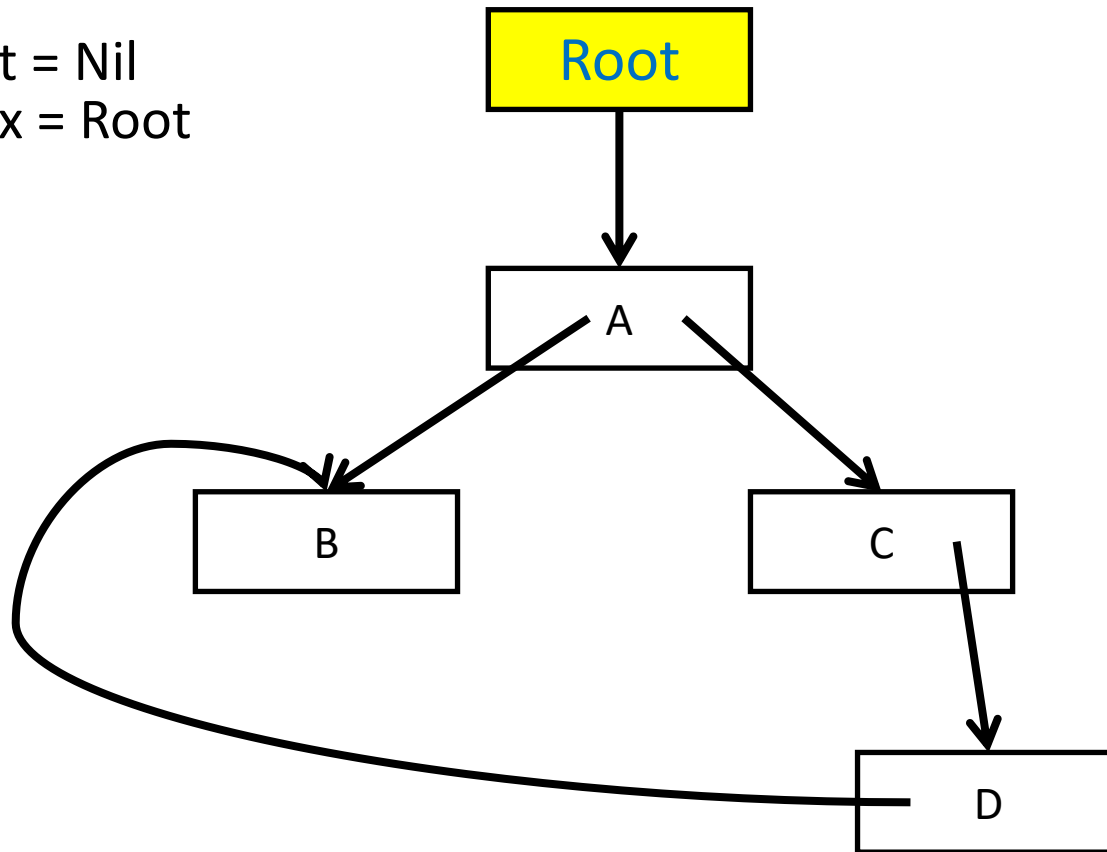
- After the contents of field *x.fi* has been pushed on the stack, the algorithm will never again look the original location of *x.fi*
- *x.fi* can be used to store one element of the stack
- When the stack is popped, restore the original value of *x.fi*

Pointer Reversal

- **Solution:** Reuse the graph components to assist backtracking
- When a new record x is encountered during the search
 - Mark record x
 - Before DFS($x.fi$), change $x.fi$ to point back to the DFS parent record of record x (pointer reversal)
 - When we can go no deeper, return, following the back links, restoring the links

Pointer Reversal

t = Nil
x = Root

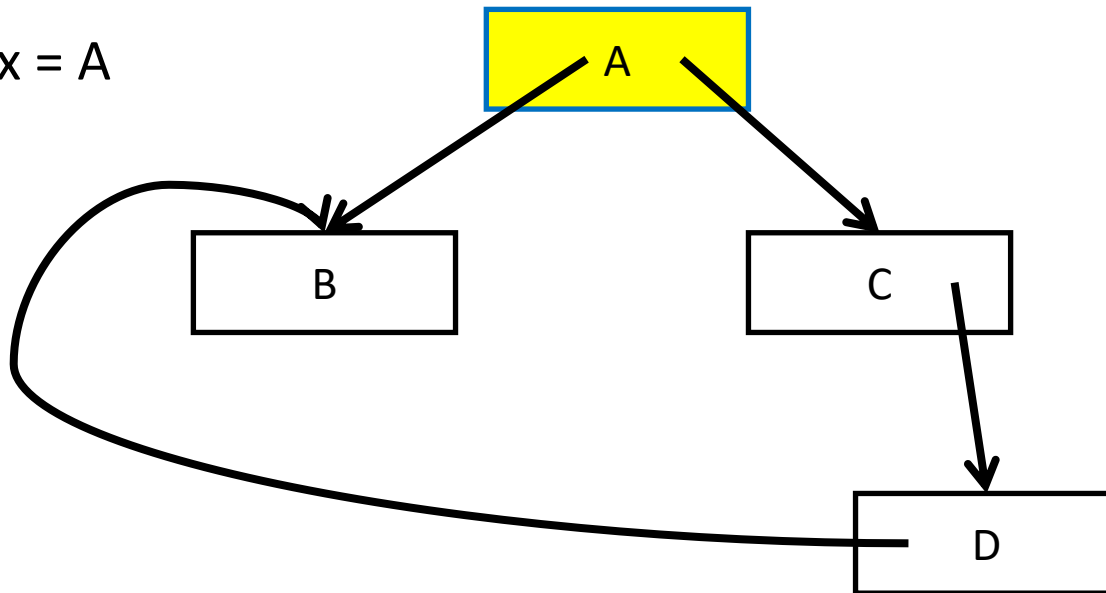


Pointer Reversal

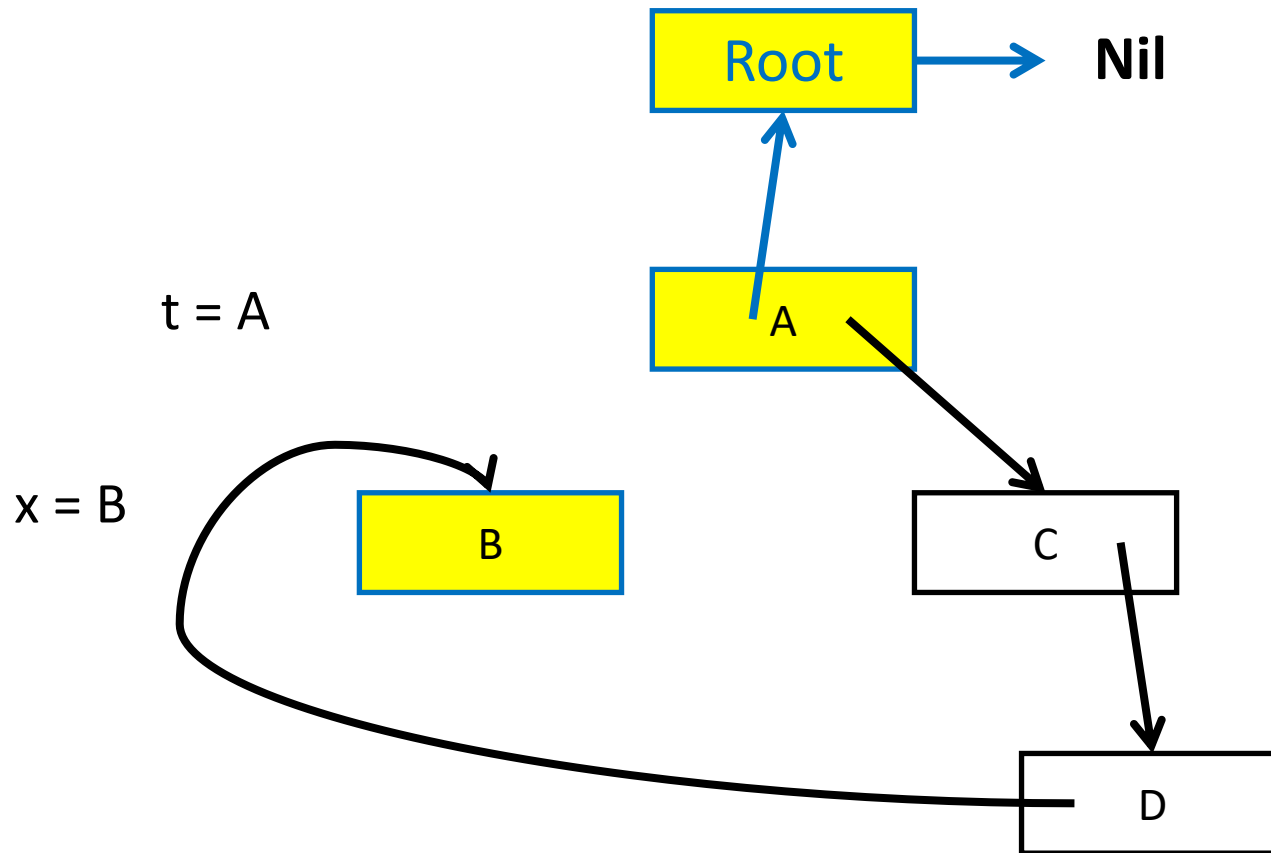
t = Root



x = A

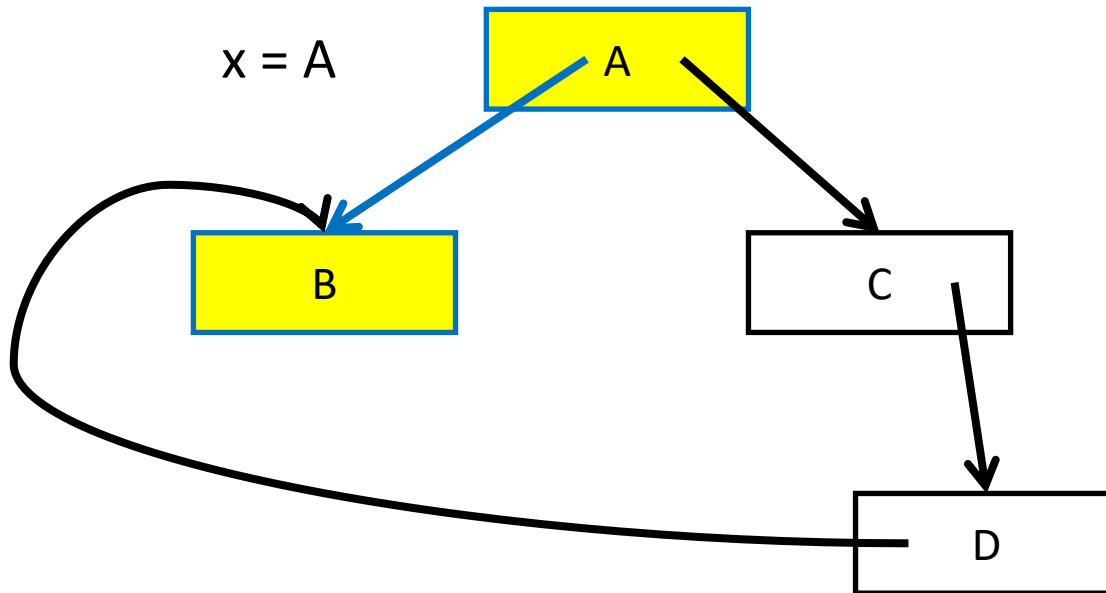


Pointer Reversal

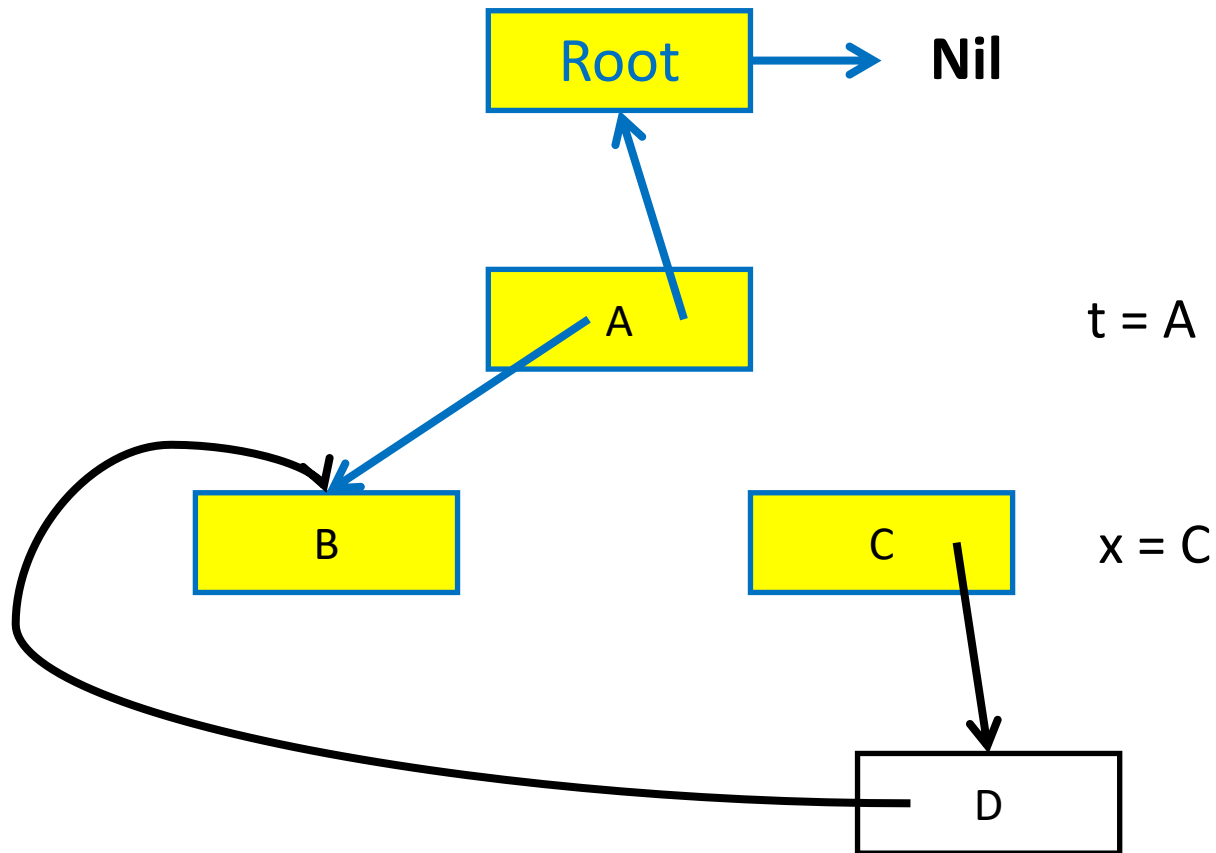


Pointer Reversal

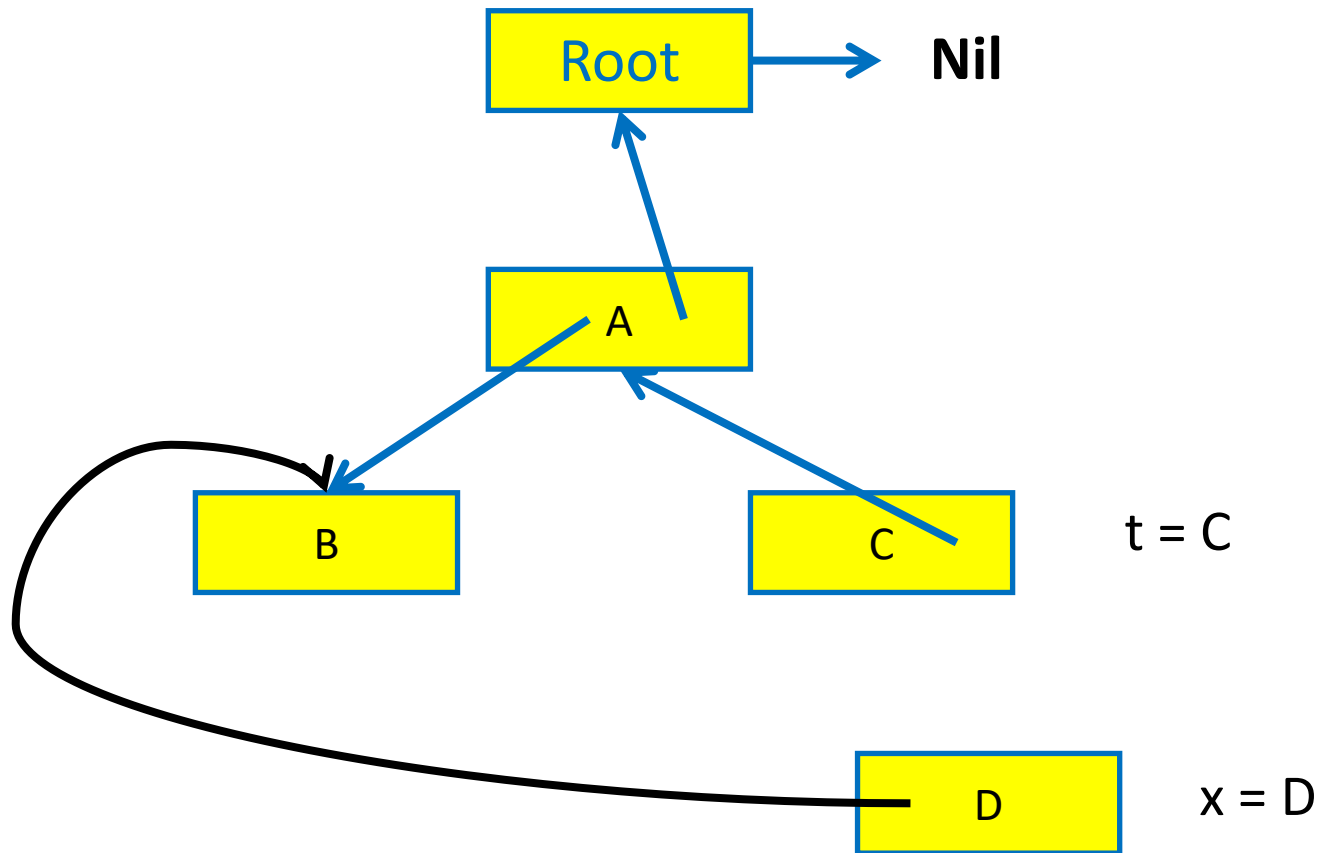
t = Root **Root** → Nil



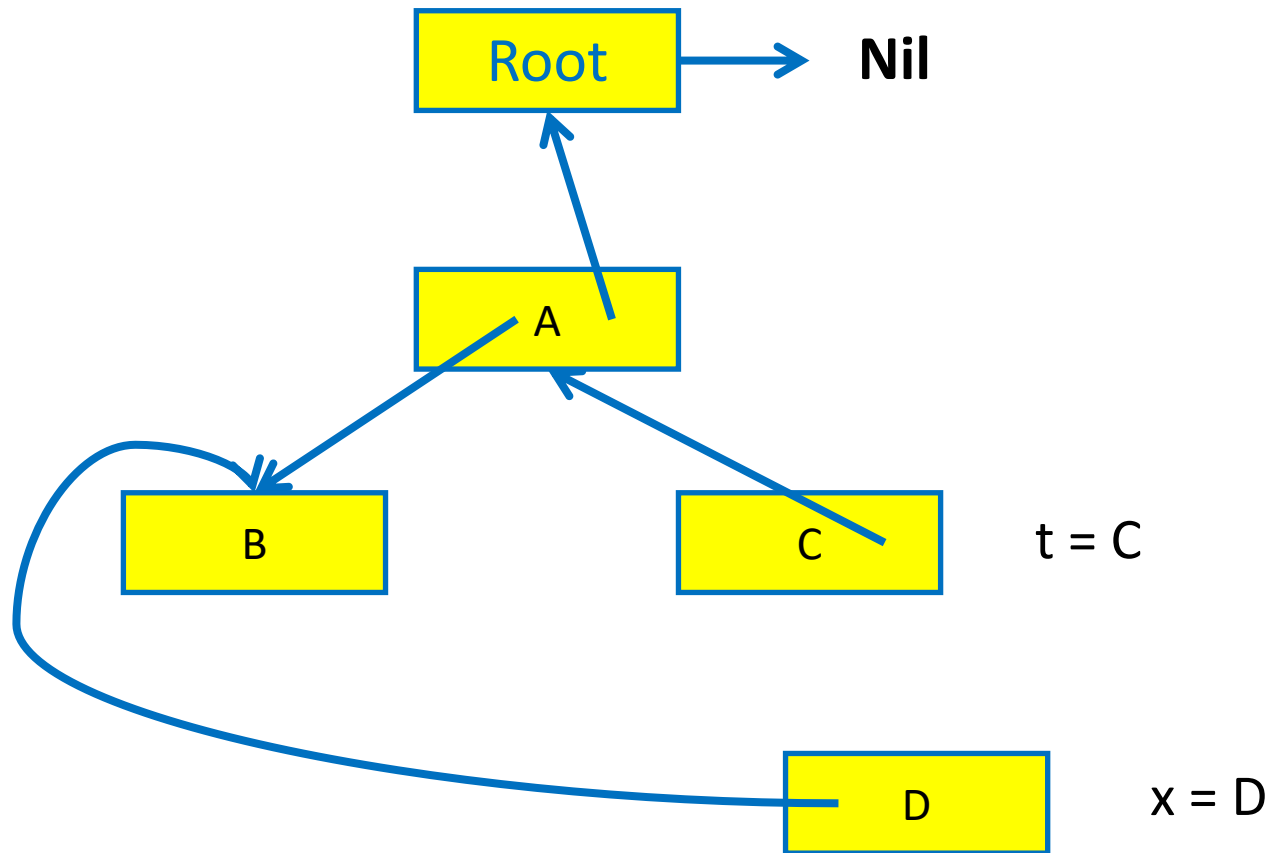
Pointer Reversal



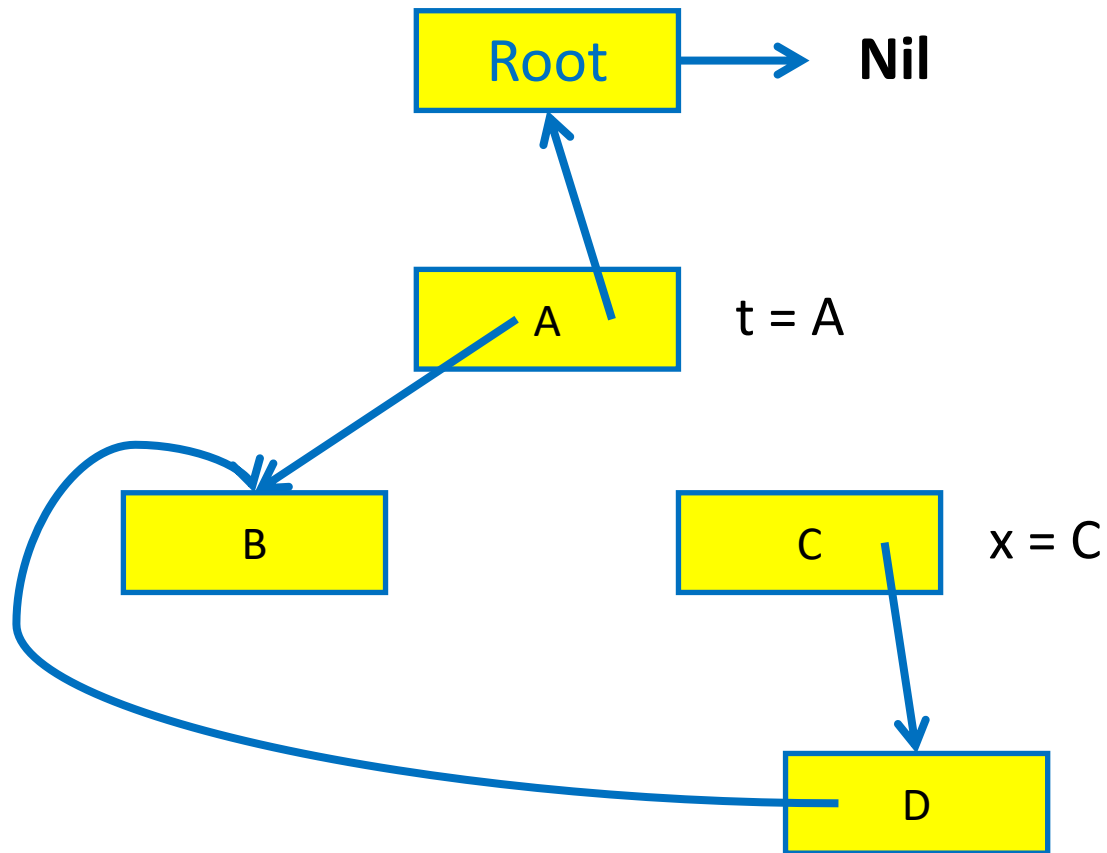
Pointer Reversal



Pointer Reversal



Pointer Reversal

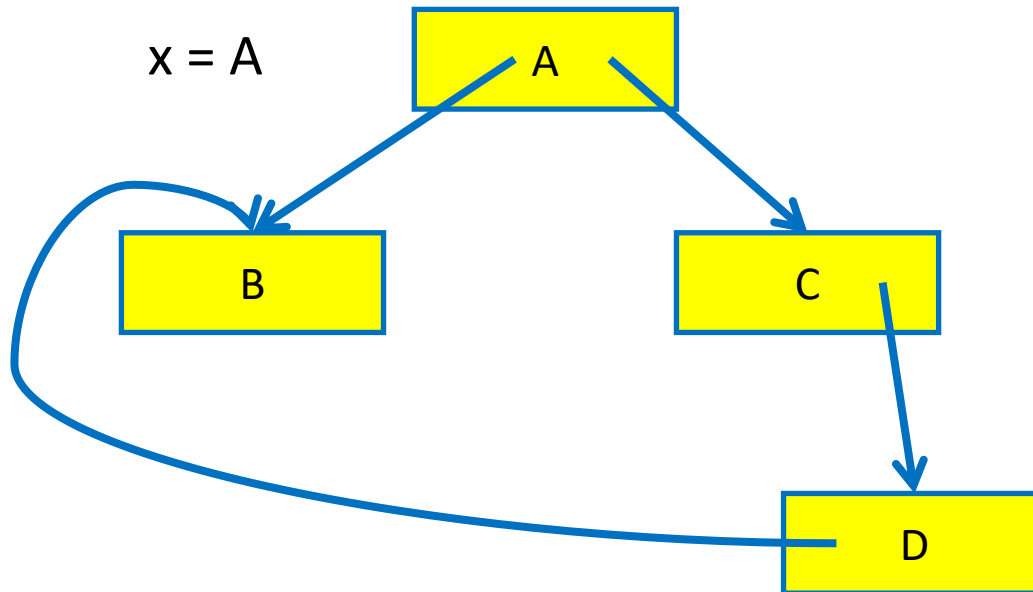


Pointer Reversal

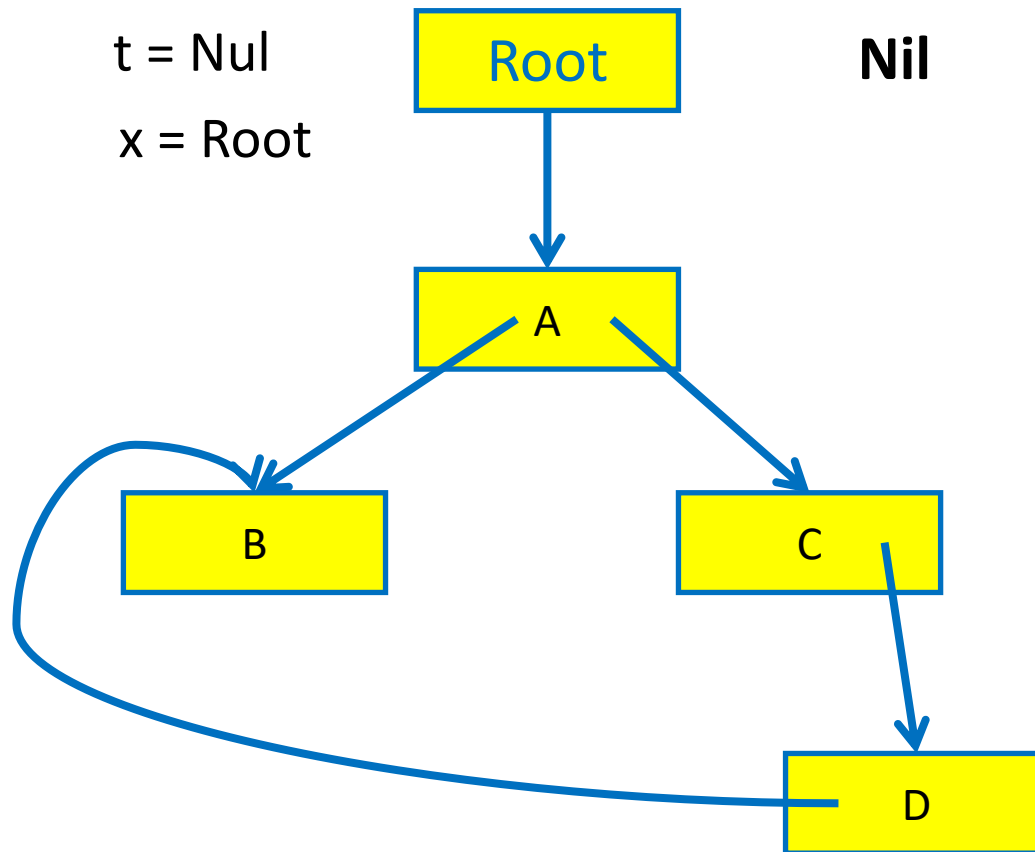
t = Root



x = A



Pointer Reversal



Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

/ initialization */*

while true

...

if $i < \#$ of fields in record x

/ process the i _th field */*

else

/ decide termination*

& back-track to the DFS parent record of x ./*

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$

- **done**: how many fields in each record have been processed
- **t**: the top of the stack

To better understand, suppose y points to record1

// let $x.fi$ point to the old top of stack (its DFS parent) ; Push x to the stack; Start to process the stored $x.fi$

// store x to y , pop the top of the stack to x

// back to the root node, terminate

// restore the correct link

// start to process the next field of the DFS parent

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

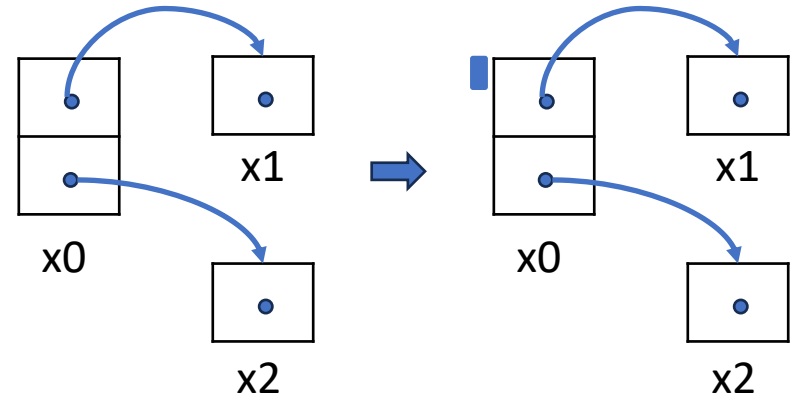
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x_0$

mark x_0

$\text{done}[x_0] = 0$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

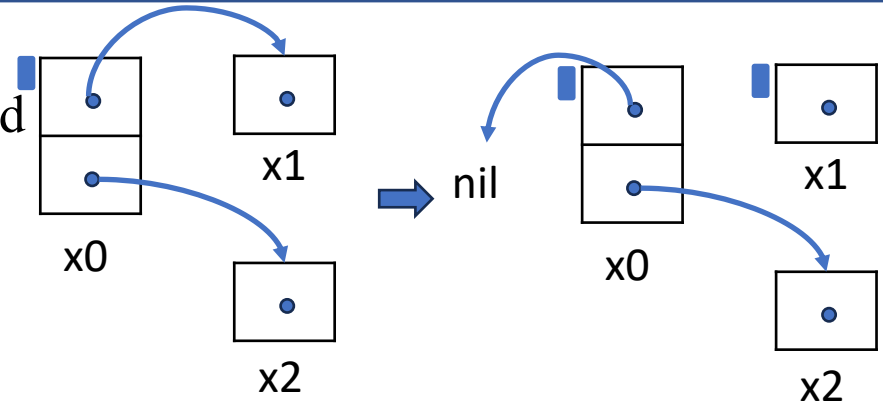
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x0$

$i = \text{done}[x0] = 0$

$y = x0.fi = x1$

$x0.fi = t = \text{nil}$

$t = x = x0$

$x = y = x1$

mark $x1$

$\text{done}[x1] = 0$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

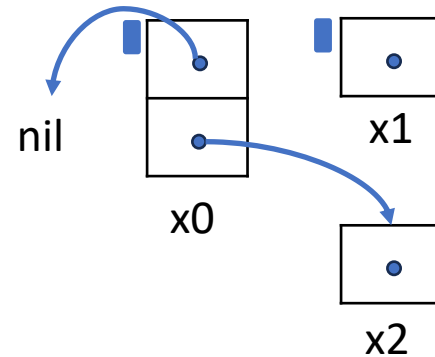
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x_0$

$x = x_1$

$i = \text{done}[x_1] = 0$

$y = x_1.f_0$, not pointer

$\text{done}[x_1] = i + 1 = 1$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

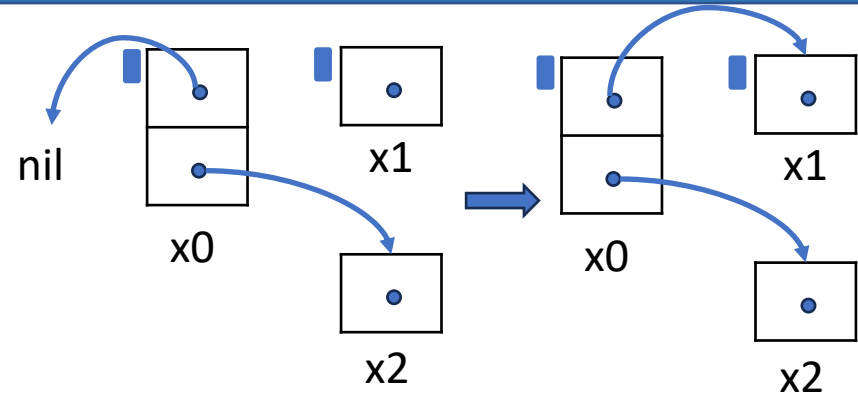
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x0$

$x = x1$

$i = \text{done}[x1] = 1$

$y = x1, x = x0$ // back to parent

$i = \text{done}[x0] = 0$

$t = x0.f_0 = \text{nil}$ // update stack top

$x0.f_0 = x1$ // restore

$\text{done}[x0] = 1$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

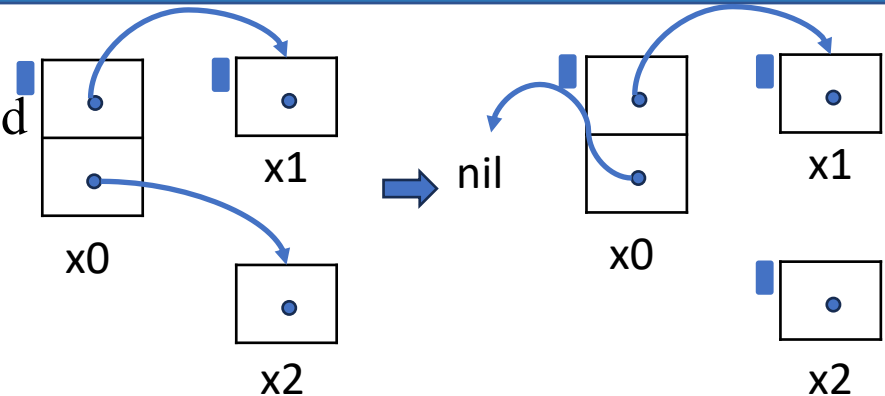
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x0$

$i = \text{done}[x0] = 1$

$y = x0.f_1 = x2$

$x0.f_1 = t = \text{nil}$

$t = x = x0$

$x = y = x2$

mark $x2$

$\text{done}[x2] = 0$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

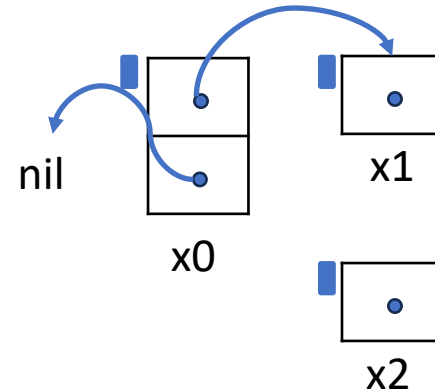
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x0$

$x = x2$

$i = \text{done}[x2] = 0$

$y = x2.f_0$, not a pointer

$\text{done}[x2] = 1$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.f_i$

if y is a pointer and record y is not marked

$x.f_i \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

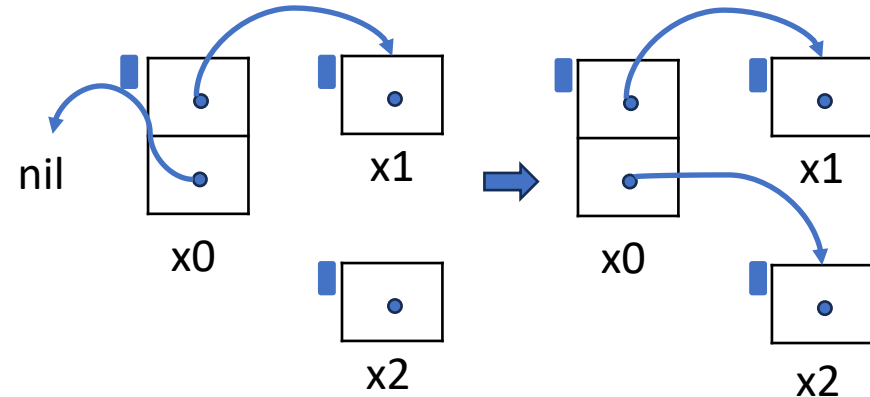
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.f_i$; $x.f_i \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = x0$

$x = x2$

$i = \text{done}[x2] = 1$

$y = x = x2$

$x = t = x0$

$i = \text{done}[x0] = 1$

$t = x0.f1 = \text{nil}$

$x0.f1 = y = x2$ // restore

$\text{done}[x0] = 2$

Pointer Reversal

function DFS(x)

if x is a pointer and record x is not marked

$t \leftarrow \text{nil}$

mark x ; $\text{done}[x] \leftarrow 0$

while true

$i \leftarrow \text{done}[x]$

if $i < \#$ of fields in record x

$y \leftarrow x.fi$

if y is a pointer and record y is not marked

$x.fi \leftarrow t$; $t \leftarrow x$; $x \leftarrow y$

mark x ; $\text{done}[x] \leftarrow 0$

else

$\text{done}[x] \leftarrow i + 1$

else

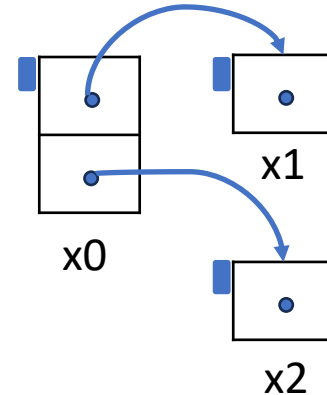
$y \leftarrow x$; $x \leftarrow t$

if $x = \text{nil}$ **then return**

$i \leftarrow \text{done}[x]$

$t \leftarrow x.fi$; $x.fi \leftarrow y$

$\text{done}[x] \leftarrow i + 1$



$t = \text{nil}$

$x = x0$

$i = \text{done}[x0] = 2$

$y = x = x0$

$x = t = \text{nil}$

return

Summary: Mark-and-Sweep

- **Pros**

- Objects/records are not moved during GC
- Be able to handle cyclic references

- **Cons**

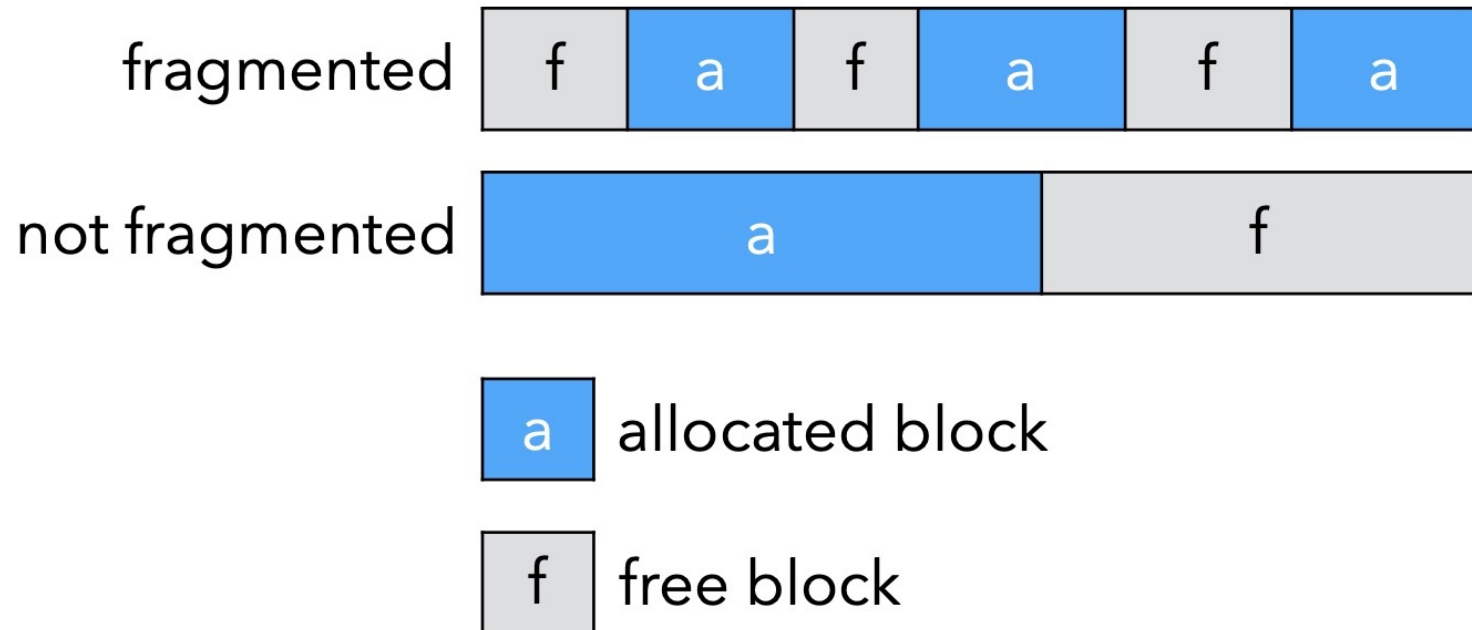
- Normal execution must be suspended
- Leads to **fragmentation** in the heap
 - Cache misses, more complex allocation

About Fragmentation

- External fragmentation:
 - The program wants to allocate a record of size n , and there are many free records smaller than n but none of the right size
- Internal fragmentation:
 - The program uses a too-large record without splitting it, so that the unused memory is inside the record instead of outside.

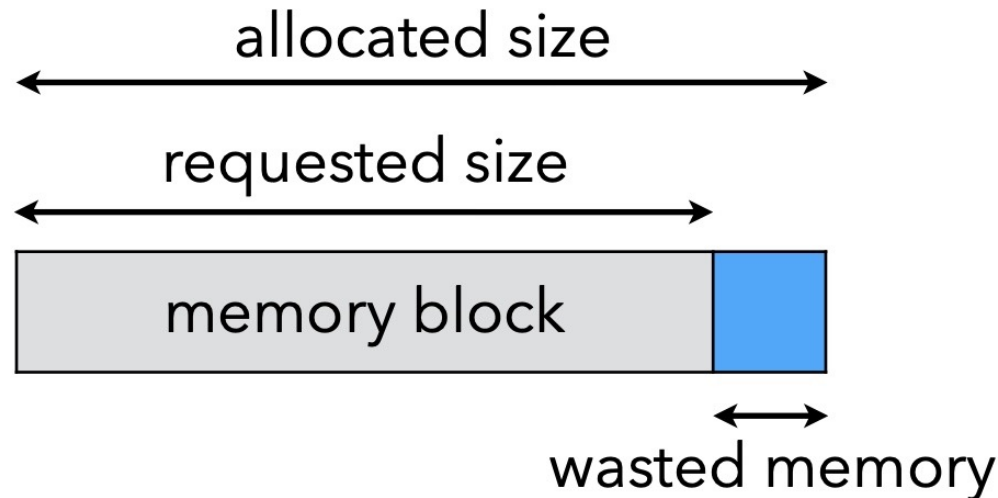
External Fragmentation: Example

- The two heaps below have the same amount of free memory, but the first suffers from external fragmentation, while the second does not.
- Therefore, some requests can be fulfilled by the second but not the first.



Internal Fragmentation: Example

- The memory manager sometimes allocates more memory than requested
 - E.g., to satisfy alignment constraints.
- This results in small amounts of wasted memory scattered in the heap, and is called internal fragmentation.



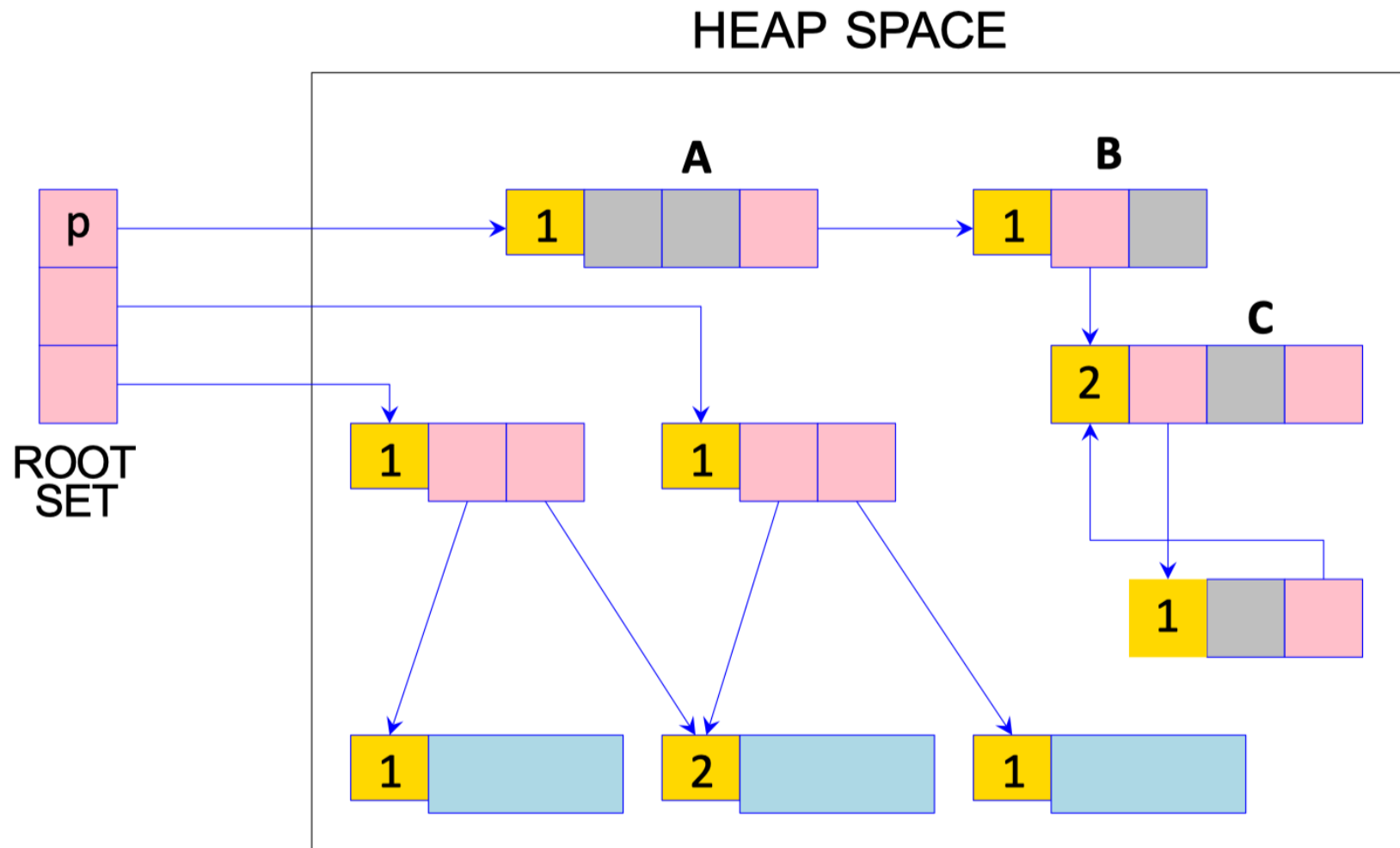
Outline

- Mark-and-Sweep Collection
- **Reference Counting**
- Copying Collection
- Interface to the Compiler

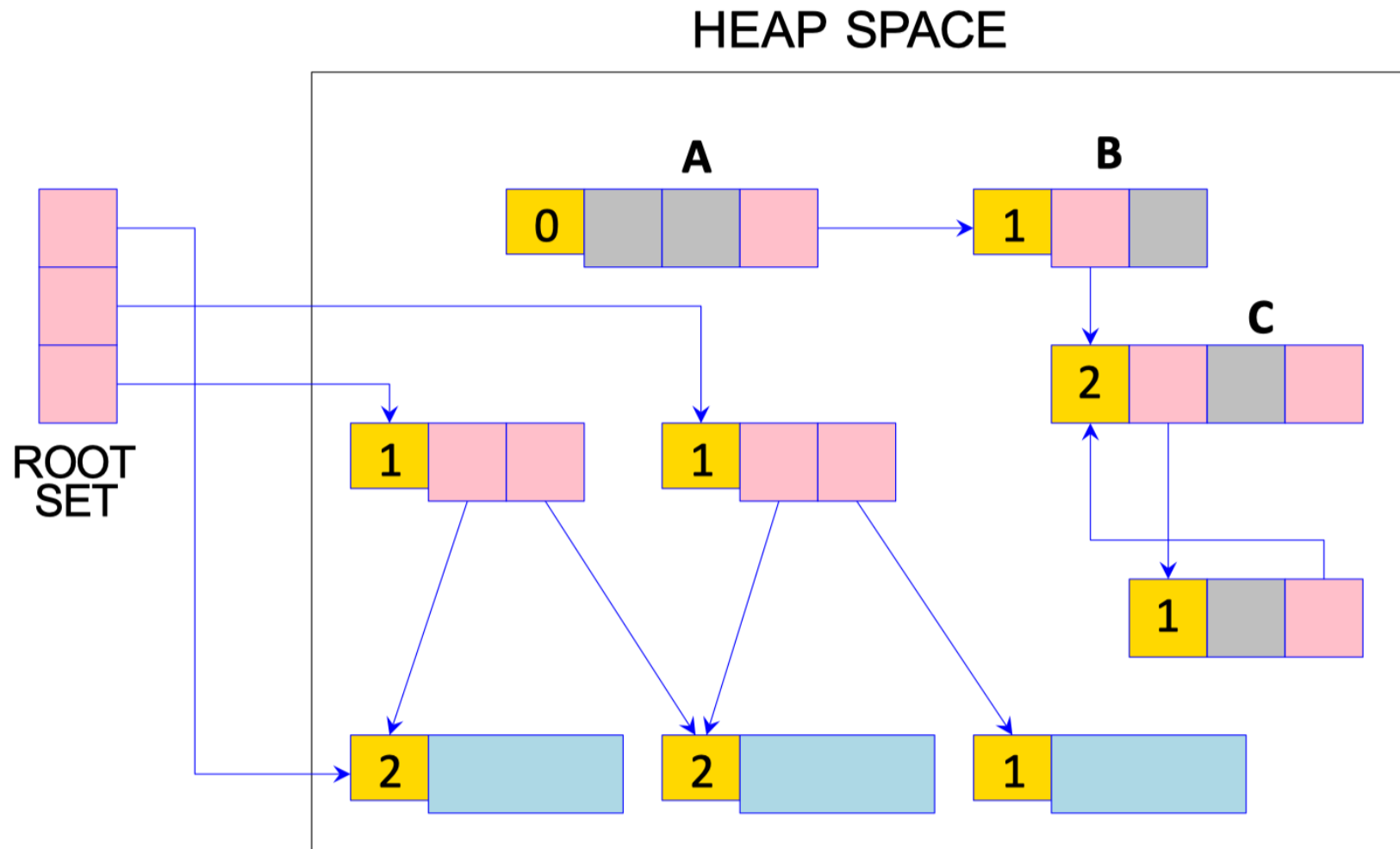
Reference Counts

- **Idea:** rather than wait for memory to be exhausted, try to collect a record when there are no more pointers pointing to it (not reachable):
 - Keep track of how many pointers point to each record (the **reference count** of this record)
 - **Store** the reference count **with each record**.
 - Whenever a new reference to the record is established, increment its reference count.
 - When the reference count goes to 0, the record is unreachable garbage, and thus can be collected.

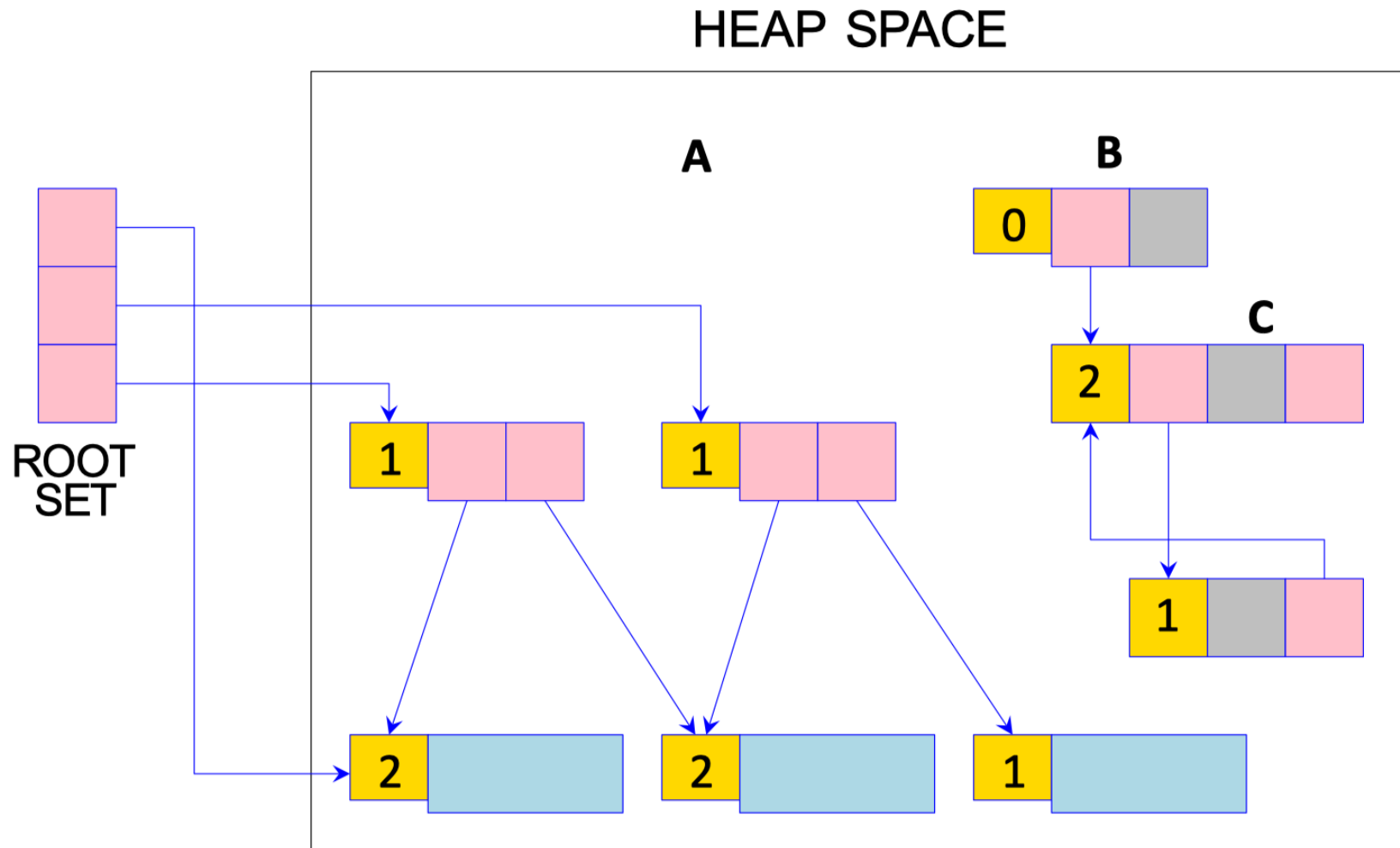
Reference Counts: Example



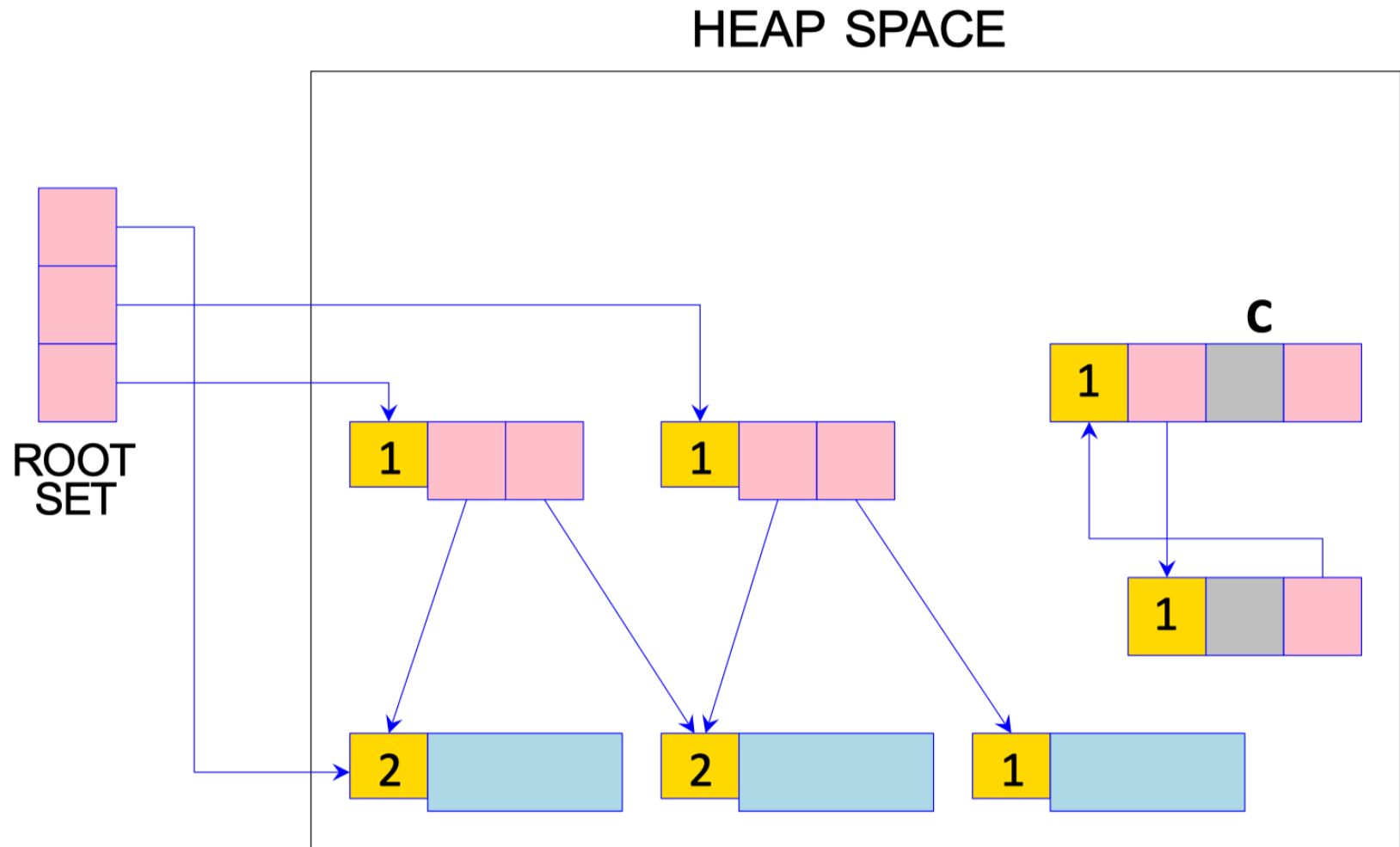
Reference Counts: Example



Reference Counts: Example



Reference Counts: Example



Reference Counts

How to keep track?

- The compiler emits extra instructions to manipulate the reference counts with each assignment operation
- **Whenever p is stored into $x.fi$ (i.e., $x.fi = p$)**
 - The reference count of p is incremented, and
 - The reference count of what $x.fi$ previously pointed to is decremented
- **If the reference count of some record r reaches zero**
 - r is put on the *freelist*, and
 - All the other records that r points to have their reference counts decremented.

Reference Counts

How to keep track?

- The compiler emits extra instructions to manipulate the reference counts with each assignment operation

$x.fi \leftarrow p$



```
z      ← x.fi
c      ← z.count
c      ← c - 1
z.count ← c
if c = 0 call putOnFreelist
x.fi   ← p
c      ← p.count
c      ← c + 1
p.count ← c
```

Reference Counts

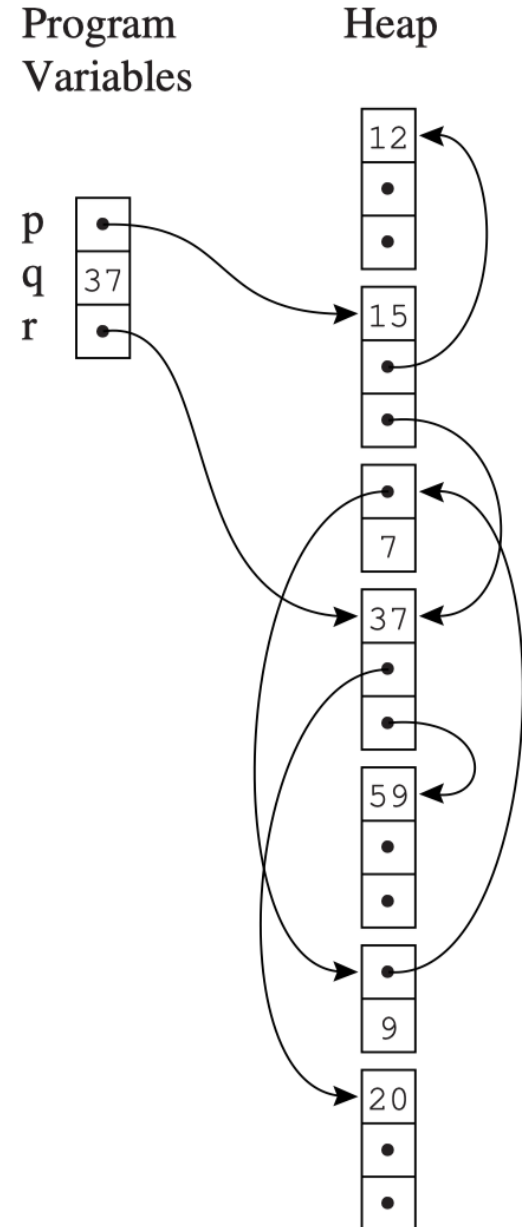
- Instead of decrementing the counts of the record pointed by **r.fi** **when record r is put on the freelist**, it is better to do this “recursive” decrementing **when r is removed from the freelist**, for two reasons:
 - It breaks up the “recursive decrementing” work into **shorter pieces**, so that the program can run more smoothly (this is important for interactive or real-time programs).
 - e.g., **r.fi -> p, p.fi -> q**
 - The recursive decrementing will be done only in one place, **in the allocator**.

Reference Counts: Problems

- Reference counting seems simple and attractive.
- But there are two major problems:
 - Cycles of garbage cannot be claimed
 - Incrementing the reference counts is very expensive indeed.

Problem 1: Reference Cycle

- A **reference cycle** is a set of objects that cyclically refer to one another.
 - e.g., the record storing 7 and the record storing 9
- Reference count tracks the number of references, not number of reachable references.
- Major problems in languages/systems that use reference counting:
 - e.g., Perl, Firefox 2.



Problem 2: Cost

- Incrementing the reference counts is very expensive indeed.
- In place of the single machine instruction $x.fi \leftarrow p$, the program must execute:

```
z      ← x.fi
c      ← z.count           // Decrease the reference count of
c      ← c - 1             what x.fi previously pointed to
z.count ← c
if c = 0 call putOnFreelist
x.fi   ← p
c      ← p.count           // Increase the reference count of p
c      ← c + 1
p.count ← c
```

Dataflow analysis can eliminate some increments and decrements, but many remain.

Analysis of Reference Counts

- **Pros:**

- Simple to implement
- Immediate collection
 - i.e., reduce the time between the object becoming garbage and its reclamation
- Incremental collection
 - cell management interleaved with program execution; no “stop-and-collect” effect

- **Cons:**

- Fails to reclaim all unreachable objects
- Can be slow if a large collection is initiated
- Noticeably slows down assignments

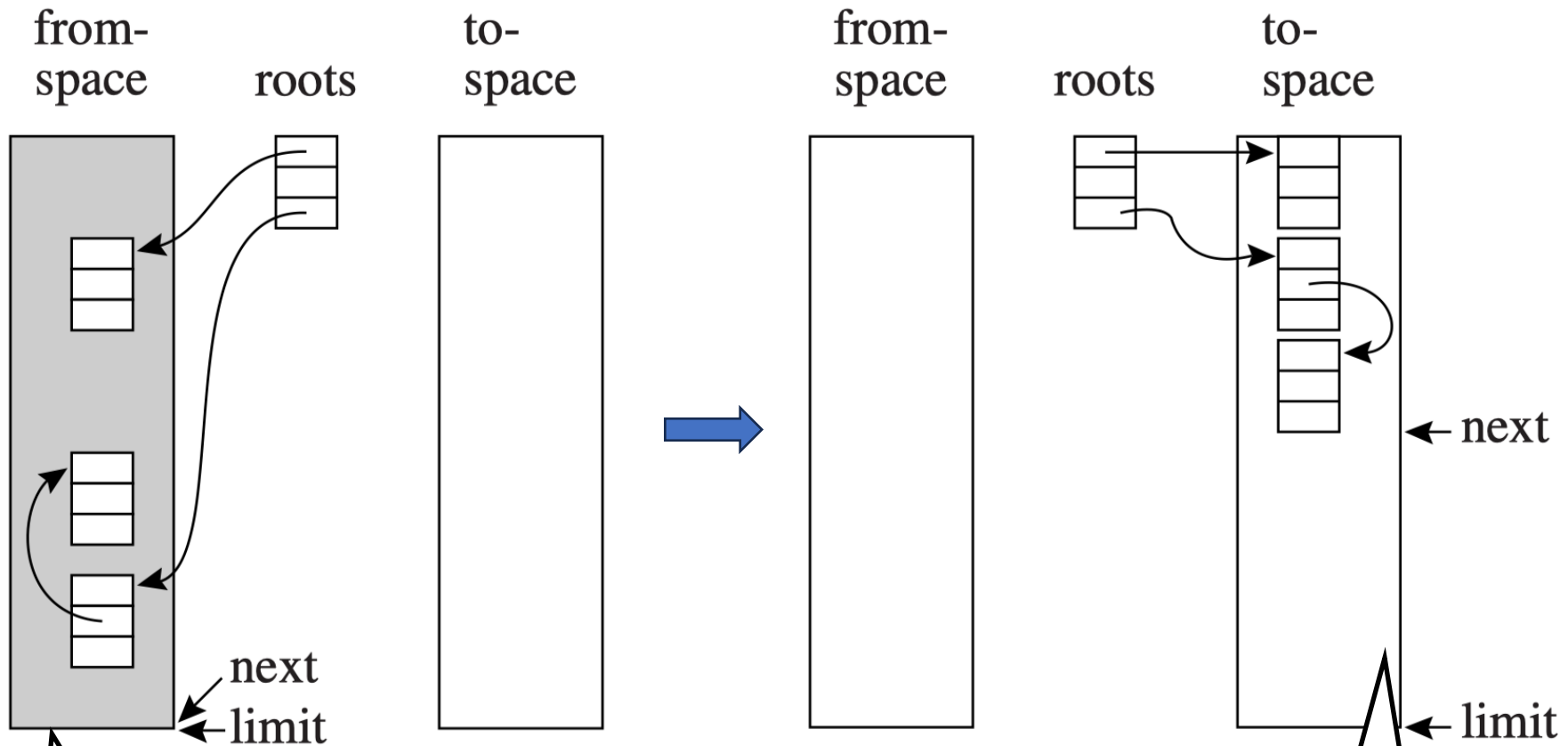
Outline

- Mark-and-Sweep Collection
- Reference Counts
- **Copying Collection**
 - Pointer Forwarding
 - Cheney's Algorithm
- Interface to the Compiler

Copying Collection

- **Basic idea:** split the memory into two parts and collect by copying
 - **from-space:** the one used by the program
 - **to-space:** the one unused until garbage collection time
- Copying garbage collection:
 - When **from-space** is exhausted, traverse the graph formed with program variables and **from-space**, and copy all reachable records to **to-space**
 - When copying is done, the roots are made to point at the **to-space** copy; the entire **from-space** is unreachable; we change the role of **from-space** and **to-space**
- The to-space copy is **compact**:
 - occupying contiguous memory without fragmentation.

Copying Collection



(a) Before collection

(b) After collection

No space to allocate.

- Allocating a new record **p** is easy: $p = \text{next}$, $\text{next} = \text{next} + n$.
- Does not have a fragmentation problem.

available to allocate

Outline

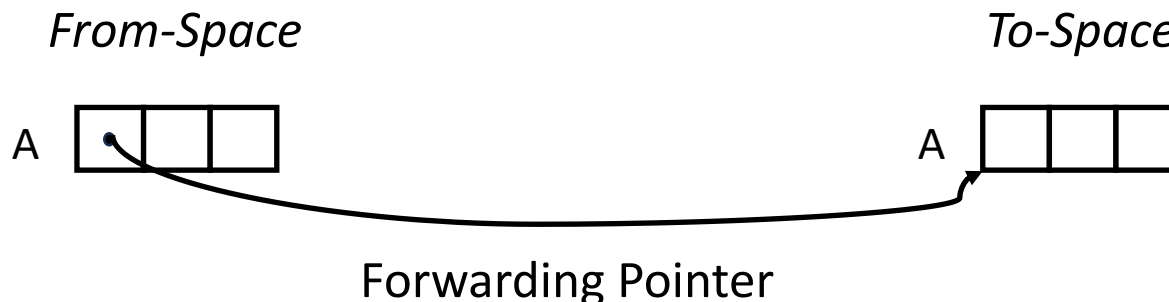
- Mark-and-Sweep Collection
- Reference Counts
- **Copying Collection**
 - **Pointer Forwarding**
 - Cheney's Algorithm
- Interface to the Compiler

Pointer Forwarding: Why

- To implement Copying Collection:
 - We need to traverse all the reachable records, as for the mark-and-sweep approach
 - As we find a reachable record, we copy it into **to-space**
- Note: we have to preserve the points-to-relations
 - Update **ALL** pointers pointing to the record.
- Suppose **B.fi** $\rightarrow$ **A**, how can we update **B.fi** to make point to the new copy of **A** when performing graph traversal (in the runtime)?

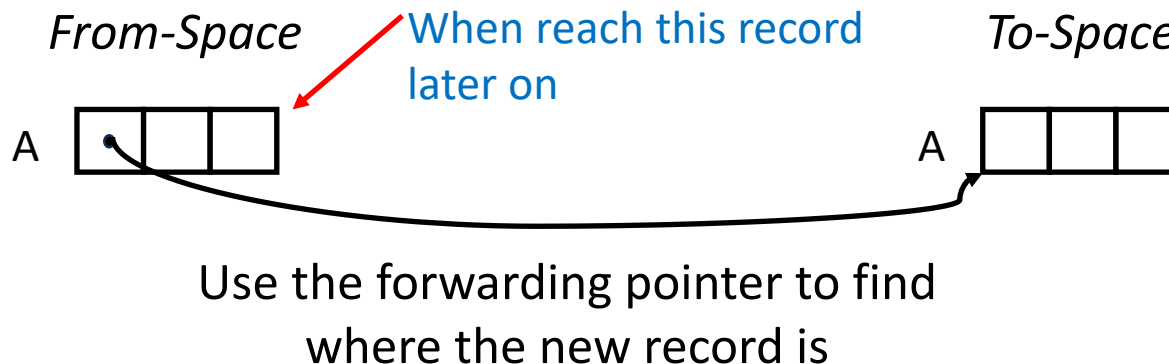
Pointer Forwarding: Insight

- Note: we have to preserve the points-to-relations
 - Update **ALL** pointers pointing to the record.
- Suppose $B.fi \rightarrow A$, how can we update $B.fi$ to make point to the new copy of A when performing graph traversal (in the runtime)?
- As we copy record A , we store in its old copy (in **from-space**) a **forwarding pointer** to its new copy



Pointer Forwarding: Insight

- Note: we have to preserve the points-to-relations
 - Update **ALL** pointers pointing to the record.
- Suppose $B.fi \rightarrow A$, how can we update $B.fi$ to make point to the new copy of A when performing graph traversal (in the runtime)?
- As we copy record A , we store in its old copy (in **from-space**) a **forwarding pointer** to its new copy
 - When we later reach a record with a forwarding pointer, we know whether it has been copied and where it is now.



Copying Collection with Pointer Forwarding

- The pointer **next** is initialized to point at the beginning of **to-space**
- **Forwarding**
 - Given a pointer **p** that points to **from-space**, make **p** point to **to-space**

```
function Forward(p)
```

```
  if p points to from-space
```

```
  then if p. f1 points to to-space
```

```
    then return p. f1
```

```
    else for each field  $f_i$  of p
```

```
      next.  $f_i \leftarrow$  p.  $f_i$ 
```

```
      p. f1  $\leftarrow$  next
```

```
      next  $\leftarrow$  next + size of record p
```

```
    return p. f1
```

```
  else return p
```

1. has already been copied
p.f1: forwarding pointer that
indicates where the copy is

Copying Collection with Pointer Forwarding

- The pointer **next** is initialized to point at the beginning of **to-space**
- **Forwarding**
 - Given a pointer **p** that points to **from-space**, make **p** point to **to-space**

```
function Forward(p)
  if p points to from-space
  then if p. f1 points to to-space
    then return p. f1
    else for each field fi of p
      next. fi ← p. fi
      p. f1 ← next
      next ← next + size of record p
    return p. f1
  else return p
```

1. has already been copied
p.f1: forwarding pointer that indicates where the copy is

2. not yet been copied

Copying Collection with Pointer Forwarding

- The pointer **next** is initialized to point at the beginning of **to-space**
- **Forwarding**
 - Given a pointer **p** that points to **from-space**, make **p** point to **to-space**

```
function Forward(p)
  if p points to from-space
  then if p. f1 points to to-space
    then return p. f1
    else for each field fi of p
      next. fi ← p. fi
      p. f1 ← next
      next ← next + size of record p
    return p. f1
  else return p
```

1. has already been copied
p.f1: forwarding pointer that indicates where the copy is

2. not yet been copied

3. not a pointer or points outside *from-space*

Outline

- Mark-and-Sweep Collection
- Reference Counts
- **Copying Collection**
 - Pointer Forwarding
 - **Cheney's Algorithm**
- Interface to the Compiler

Cheney's Algorithm

- **Cheney's algorithm:** a collection algorithm using breadth-first search (BFS) to traverse the reachable data.
- How to maintain the worklist for BFS?
- Introduce a pointer **scan**, and partition **to-space** into three contiguous regions with **scan** and **next**
 - **Copied:** the record is copied, but we haven't looked at pointers inside the record
 - **Copied and scanned:** the record is copied and we have handled all the pointers in the record.
 - Empty



Cheney's Algorithm

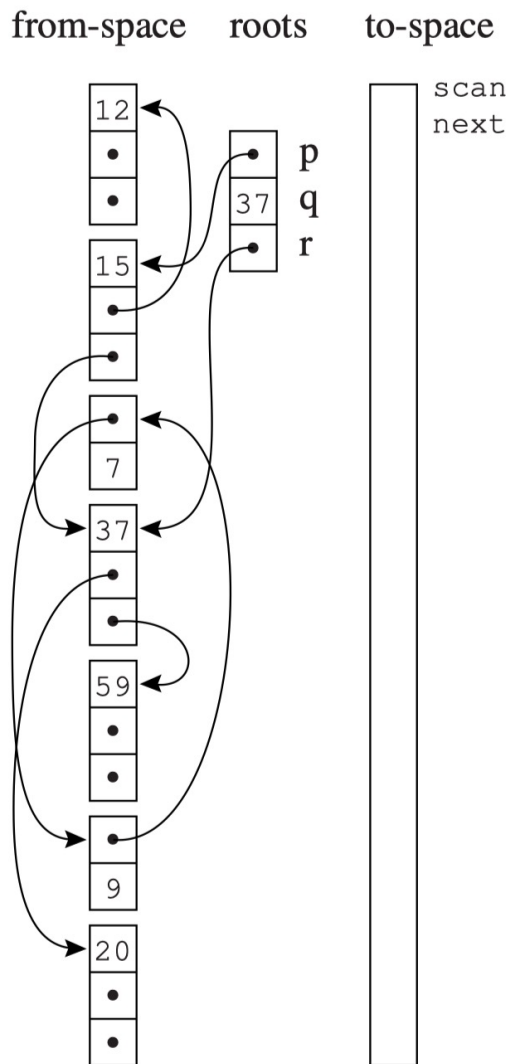
- **Cheney's algorithm:** a collection algorithm using breadth-first search to traverse the reachable data.

ALGORITHM 13.9: Breadth-first copying garbage collection

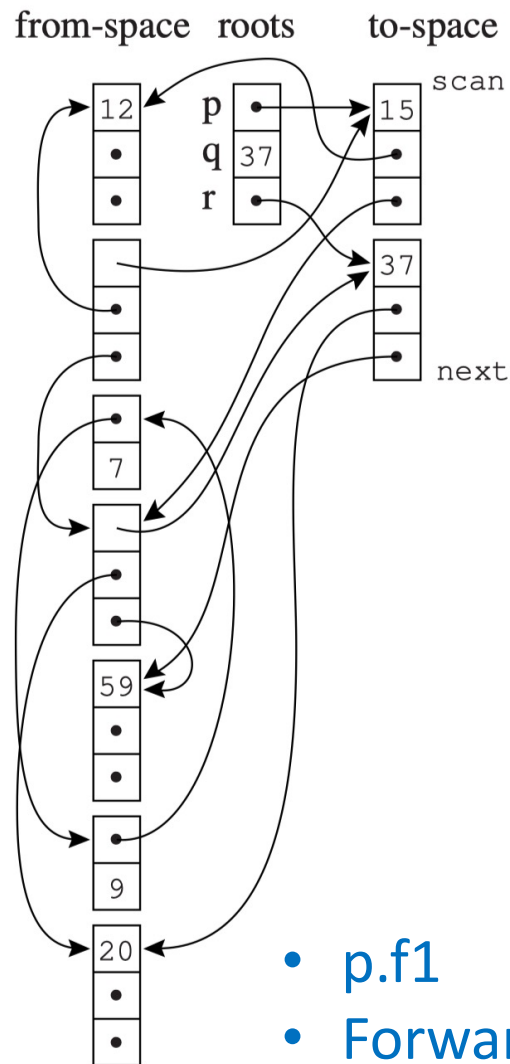
```
scan  $\leftarrow$  next  $\leftarrow$  beginning of to-space  
for each root r  
    r  $\leftarrow$  Forward(r) // increase next, but keep scan unchanged  
while scan < next  
    for each field fi of record at scan  
        scan.fi  $\leftarrow$  Forward(scan.fi)  
    scan  $\leftarrow$  scan + size of record at scan
```



Breadth-first Copying Collection

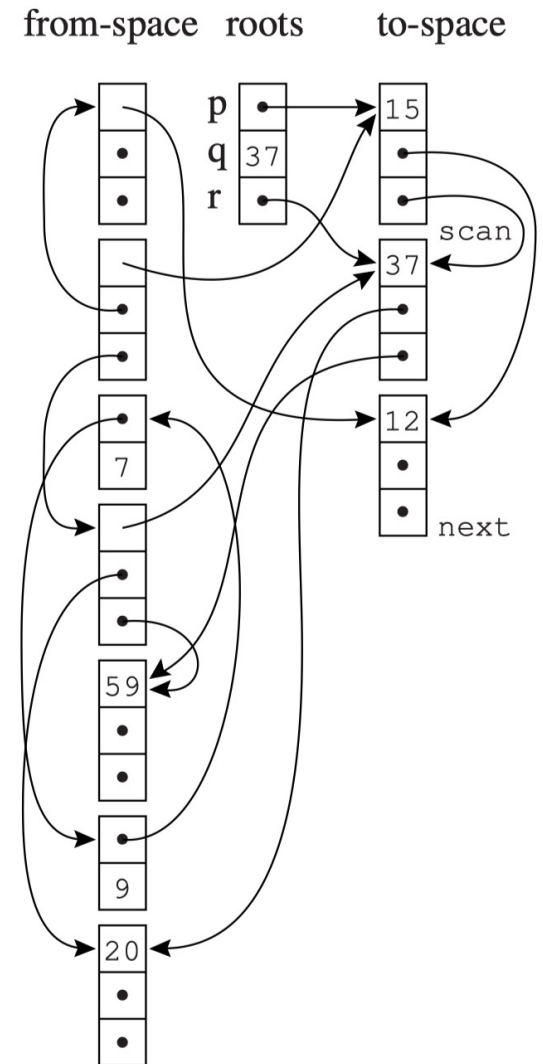


(a) Before collection



(b) Roots forwarded

- p.f1
- Forward



(c) One record scanned

Limitation of Cheney's Algorithm: Locality of Reference

- In a computer system with virtual memory, or with a memory cache, good locality of reference is important :
 - After the program fetches address a , then the memory subsystem expects addresses near a to be fetched soon.
- Pointer data structures copied by breadth-first have poor locality of reference.
 - If a record at address a points to another record at address b , it is likely that a and b will be far apart.
 - Why?
- Depth-first copying gives better locality
 - But depth-first copy requires pointer-reversal, which is inconvenient and slow.

A Hybrid Algorithm

- A hybrid, partly depth-first and partly breadth-first algorithm can provide acceptable locality.

```
function Forward(p)
  if p points to from-space
  then if p.fl points to to-space
    then return p.fl
    else Chase(p); return p.fl
  else return p
```

- The basic idea is to use breadth-first copying, but whenever an object is copied, see if some child can be copied near it.

```
function Chase(p)
  repeat
    q ← next
    next ← next + size of record p
    r ← nil
    for each field fi of record p
      q.fi ← p.fi
      if q.fi points to from-space and
      q.fi.fl does not point to to-space
        then r ← q.fi
    p.fl ← q
    p ← r
  until p = nil
```

Summary: Copying Collection

- **Pros:**

- Simplicity: no stack or pointer reversal required
- Run-time proportional to `#reachable_objects`
- Leave free space contiguous
 - Automatic compaction eliminates fragmentation

- **Cons:**

- Half of memory is wasted
- Poor locality (at least for the Cheney's algorithm)
- Precise type information required (Pointer or not)

Outline

- Mark-and-Sweep Collection
- Reference Counts
- Copying Collection
- *Generational Collection*
- *Incremental Collection*
- **Interface to the Compiler**

Interface to the Compiler

Although the garbage collector is a part of “runtime”, the compiler for a garbage-collected language interacts with the garbage collector by:

- generating code that allocate records
- describing locations of roots for each garbage collection cycle
- describing the layout of data records on the heap
- generating instructions to implement a read or write barrier (for some versions of incremental collection)
- ...

Fast Allocation

- Allocation matters for programs:
 - Empirical measures: one in seven instructions is a **store**
 - I.e., we have at most $1/7$ word of allocation per instruction
- There is a considerable cost to create the heap records.
- **Copying collection** should be used (as it is fast)
 - the allocation space is contiguous free region
 - the next free location is **next** and
 - the end of the region is **limit**

Fast Allocation

The steps to allocate a record of size N:

1. Call the allocate function
 2. Test $next + N < limit$? (If the test fails, call GC)
 3. Move $next$ to $result$
 4. Clear $M[next]$, $M[next+1]$, ..., $M[next + N - 1]$
 5. $next \leftarrow next + N$
 6. Return from the allocate function
-
- A. Move result into some computationally useful place
 - B. Store useful values into the record
- (Steps A and B are not allocation overhead)

Fast Allocation

Step 1 and 6 should be eliminated by *inline expanding* the allocate function

The steps to allocate a record of size N:

- ~~1. Call the allocate function~~
 2. Test $next + N < limit$? (If the test fails, call GC)
 3. Move *next* to *result*
 4. Clear $M[next], M[next+1], \dots, M[next + N - 1]$
 5. $next \leftarrow next + N$
 - ~~6. Return from the allocate function~~
- A. Move result into some computationally useful place
- B. Store useful values into the record
- (Steps A and B are not allocation overhead)

Fast Allocation

Step 3 can often be eliminated by combining it with Step A.

The steps to allocate a record of size N :

- ~~1. Call the allocate function~~
 2. Test $next + N < limit$? (If the test fails, call GC)
 - A. Move $next$ into some computationally useful place
 4. Clear $M[next], M[next+1], \dots, M[next + N - 1]$
 5. $next \leftarrow next + N$
 - ~~6. Return from the allocate function~~
 - ~~A. Move result into some computationally useful place~~
 - B. Store useful values into the record
- (Steps A and B are not allocation overhead)

Fast Allocation

Step 4 can be eliminated
by in factor of Step B.

The steps to allocate a record of size N:

- ~~1. Call the allocate function~~
 2. Test $next + N < limit$? (If the test fails, call GC)
 - A. Move $next$ into some computationally useful place
 - ~~4. Clear $M[next]$, $M[next+1]$, ..., $M[next + N - 1]$~~
 5. $next \leftarrow next + N$
 - ~~6. Return from the allocate function~~
 - ~~A. Move result into some computationally useful place~~
 - B. Store useful values into the record
- (Steps A and B are not allocation overhead)

Fast Allocation

Step 2 and 5 cannot be eliminated. But they can be shared among multiple allocations.

The steps to allocate a record of size N :

- ~~1. Call the allocate function~~
 2. Test $next + N < limit$? (If the test fails, call GC)
 - A. Move $next$ into some computationally useful place
 - ~~4. Clear $M[next]$, $M[next+1]$, ..., $M[next + N - 1]$~~
 5. $next \leftarrow next + N$
 - ~~6. Return from the allocate function~~
 - ~~A. Move result into some computationally useful place~~
 - B. Store useful values into the record
- (Steps A and B are not allocation overhead)

Fast Allocation

- Step 2: Test $next + N < limit$? (If the test fails, call GC)
- Step 5: $next \leftarrow next + N$
- By keeping `next` and `limit` in registers, steps 2 and 5 can be done in a total of **three instructions**
- By this combination of techniques, the cost of allocating a record – and then eventually garbage collecting it - can be brought down to **about four instructions**

Describing Data Layouts

- The collector must be able to operate on records of any types that the program has declared
 - It must be able to determine **the number of fields** in each record and **whether each field is a pointer**

```
scan ← next ← beginning of to-space  
for each root r  
  r ← Forward(r)  
while scan < next  
  for each field fi of record at scan  
    scan.fi ← Forward(scan.fi)  
  scan ← scan + size of record at scan
```

- How to know such information?

Describing Data Layouts

- How to know such information?
- For statically typed language, such as Tiger or Pascal, or for object-oriented languages, such as Java:
 - have the first word of every object **point** to a special **type- or class-descriptor** record.
- Type- or class-descriptor
 - The **total size** of the object
 - **The location of each pointer field**
- The type- or class-descriptor must be generated by the compiler from the static type information.
 - In which phase?
 - Semantic analysis

Describing Data Layouts

- For statically typed languages: an overhead of one word per record
- For object-oriented languages: no additional per-object overhead attributable to garbage collection
 - They need this descriptor-pointer to implement dynamic method lookup.

Pointer Map

- To implement garbage collectors, the compiler must identify to the collector:
 - every **pointer-containing** temporary and local variables
 - whether it is in a register or in an activation record
- Tiger's solution: building a **pointer map**
 - All maps are generated by compilers
 - Compilers know which temp is a pointer during compilation

Pointer Map

- The set of live temporaries can change at every instruction.
- The **pointer map** is different at every point in the program:
 - it is simpler to describe the pointer map only at points **where a new garbage collection can begin**.
- Where can a new garbage collection begin?
 - calls to the **alloc** function
 - any function-call
 - any function-call might be calling a function which in turn calls **alloc**

Pointer Map

The pointer map is best keyed by **return addresses**. Because the return address is what the collector will see in the very next activation record.

- To find all the roots, the collector starts at the **top** of the stack and scans downward
 - Each return address keys the pointer-map entry that describes the next frame
 - In each frame, the collector marks (or forwards, if copying collection) from the pointers in that frame
- Callee-save registers need special handling
 - Suppose function *f* calls *g*, which calls *h*.
 - The pointer map for *g* must describe which of its callee-save registers contain pointers at the call to *h* and which are “inherited” from *f*.

Derived Pointers

- Sometimes a compiled program has a pointer that points into the middle of a heap record, or that points before or after the record.
- For example:

- $a[i-2000]$ can be calculated internally as $M[a-2000+i]$

$t1 \leftarrow a-2000$

$t2 \leftarrow t1 + i$

$t3 \leftarrow M[t2]$

- If $a[i-2000]$ occurs inside a loop, the compiler might choose to hoist $t1 \leftarrow a - 2000$ outside the loop to avoid recalculating it in each iteration.
- if the loop also contains an *alloc*, and a garbage collection occurs while $t1$ is live: $t1$ does not point the the beginning of an object or (worse yet) points to an unrelated object.

Derived Pointers

$t1 \leftarrow a - 2000$

$t2 \leftarrow t1 + i$

$t3 \leftarrow M[t2]$

- We say that $t1$ is *derived* from the *base* pointer a .
- The collector will be confused by $t1$
- How to handle this problem?
- The pointer map must identify each *derived pointer* and tell its base pointer.
- When the collector relocates a to address a' , it must adjust $t1$ to point to address $t1 + a' - a$
- a must remain live as long as $t1$ is live. For example:

Derived Pointers

```
let
  var a := int array[100] of 0
in
  for i := 1930 to 1990
    do f(a[i-2000])
end
```

```
r1 ← 100
r2 ← 0
call alloc
a ← r1
t1 ← a - 2000
i ← 1930
L1 : r1 ← M[t1 + i]
call f
L2 : i ← i + 1
if i ≤ 1990 goto L1
```

- The temporary **a** appears dead after the assignment to **t1**
- But then the pointer map associated with the return address **L2** would not be able to “explain” **t1** adequately.
- Therefore, a **derived pointer implicitly keeps its base pointer live**.

Summary

- Mark-and-Sweep Collection
- Reference Counts
- Copying Collection
- Interface to the Compiler

Homework

- **13.2**